



Indexing Metric Spaces for Exact Similarity Search

LU CHEN, YUNJUN GAO, XUAN SONG, ZHENG LI, and YIFAN ZHU, College of Computer Science, Zhejiang University, China

XIAOYE MIAO, Center for Data Science, Zhejiang University, China

CHRISTIAN S. JENSEN, Department of Computer Science, Aalborg University, Denmark

With the continued digitization of societal processes, we are seeing an explosion in available data. This is referred to as big data. In a research setting, three aspects of the data are often viewed as the main sources of challenges when attempting to enable value creation from big data: volume, velocity, and variety. Many studies address volume or velocity, while fewer studies concern the variety. Metric spaces are ideal for addressing variety because they can accommodate any data as long as it can be equipped with a distance notion that satisfies the triangle inequality. To accelerate search in metric spaces, a collection of indexing techniques for metric data have been proposed. However, existing surveys offer limited coverage, and a comprehensive empirical study exists has yet to be reported. We offer a comprehensive survey of existing metric indexes that support exact similarity search: we summarize existing partitioning, pruning, and validation techniques used by metric indexes to support exact similarity search; we provide the time and space complexity analyses of index construction; and we offer an empirical comparison of their query processing performance. Empirical studies are important when evaluating metric indexing performance, because performance can depend highly on the effectiveness of available pruning and validation as well as on the data distribution, which means that complexity analyses often offer limited insights. This article aims at revealing strengths and weaknesses of different indexing techniques to offer guidance on selecting an appropriate indexing technique for a given setting, and to provide directions for future research on metric indexing.

128

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Information Systems** → **Data Management Systems**; • **Theory of Computation** → **Design and Analysis of Algorithms**;

Additional Key Words and Phrases: Metric spaces, indexing and querying, metric similarity search

ACM Reference format:

Lu Chen, Yunjun Gao, Xuan Song, Zheng Li, Yifan Zhu, Xiaoye Miao, and Christian S. Jensen. 2022. Indexing Metric Spaces for Exact Similarity Search. *ACM Comput. Surv.* 55, 6, Article 128 (December 2022), 39 pages. <https://doi.org/10.1145/3534963>

This work was supported in part by the NSFC under Grants No. (62102351, 62025206 and 61972338), the Zhejiang Provincial Natural Science Foundation under Grant No. LR21F020005 and the DIREC center project.

Authors' addresses: L. Chen, Y. Gao (corresponding author), X. Song, Z. Li, and Y. Zhu, College of Computer Science, Zhejiang University, 38 Zheda Road, Hangzhou, China, 310027; emails: {luchen, gaoyj, xsong, zhengli3401, xtf_z}@zju.edu.cn; X. Miao, Center for Data Science, Zhejiang University, 86 Yuhangtang Road, Hangzhou, China, 310058; email: miaoxy@zju.edu.cn; C. S. Jensen, Department of Computer Science, Aalborg University, Selma Lagerlöfs Vej 300, Aalborg, Denmark, 9220; email: csj@cs.aau.dk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

0360-0300/2022/12-ART128 \$15.00

<https://doi.org/10.1145/3534963>

1 INTRODUCTION

Massive and increasing volumes of data are being generated. For example, as suggested in Figure 1, global navigation satellite systems, e.g., GPS, and communication-network based positioning technologies enable the generation of massive spatio-temporal data by in-vehicle devices and smartphones. Massive multi-media data are being generated by surveillance cameras, smart-assistant microphones, and smartphones. Furthermore, massive volumes of social media data is being generated by services such as Facebook, Twitter, and WhatsApp. This proliferation of data offers opportunities for value creation, benefiting businesses as well as society. On the flipside, this state of affairs also presents difficult challenges due to the sheer volume, velocity, and variety of the data. Here, volume refers to massive data, velocity refers to the phenomenon that the data arrives at a rapid rate, and variety refers to the characteristic that the data stems from a wide range of sources and is diverse in terms of its structure and data type. Many studies and products address volume or velocity, while variety is receiving less attention. More specifically, many platforms (e.g., MapReduce [52], Hadoop,¹ Spark,² Flink,³ and Storm⁴) address volume and velocity, while only few (notably Azure Cosmos DB⁵ and GeminiDB⁶) focus on the variety aspect by supporting a range of models (such as tables, graphs, and documents). The use of metric spaces enables addressing variety because any type of data that can be associated with a distance notion that satisfies the triangle inequality can be treated as metric data. Hence, by using metric spaces, unified solutions can be developed that enable value creation from diverse data.

Due to the generality of metric spaces, search in metric spaces plays an important role in many real-life applications, with similarity search taking center stage [35, 115, 151]. Given a query object, similarity search finds similar objects according to the definition of similarity. In intelligent transportation, similarity queries can be used to find the nearest restaurant for a user. In multi-media retrieval, similarity queries can be utilized to identify images similar to a specified image. In recommender systems, similarity queries can be employed to generate personalized recommendations for users. In addition, similarity queries can be used to accelerate data mining tasks. For example, similarity search can be used as the first step in clustering [96] or outlier detection [1]. In the above applications, metric spaces can accommodate a wealth of data types (e.g., locations, images, and strings), and can be used to support a wide range of distance metrics that quantify the similarity of objects, including the shortest path distance for locations, the L_p -norm and earth mover's distance [81] for images, and the edit distance for strings. Metric spaces require no assumptions on the representations of objects, and any distance notation satisfying the triangle inequality can be accommodated.

A number of indexing techniques exist that aim to accelerate search in metric spaces. The main goal of this article is to present a comprehensive survey that describes and analyzes existing metric indexes. Although several studies [35, 66, 115, 151] offer valuable surveys of metric indexing techniques, no study has yet offered comprehensive theoretical and experimental analysis. This study extends a previous study [39] by the authors that covers on pivot-based metric indexing. We extend that study as follows: (i) we cover existing metric indexes that support exact similarity search; (ii) we summarize the partitioning methods, and improve the coverage of the pruning and validation techniques used for exact similarity search; (iii) we cover the time and space complexity

¹Apache Hadoop. <http://hadoop.apache.org/> (2008).

²Apache Spark. <http://spark.apache.org/> (2014).

³Apache Flink. <http://flink.apache.org/> (2014).

⁴Apache Storm. <http://storm.apache.org/> (2014).

⁵Microsoft. <https://azure.microsoft.com/services/cosmos-db/> (2017).

⁶Huawei. <https://www.huaweicloud.com/intl/product/geminidb.html> (2019).

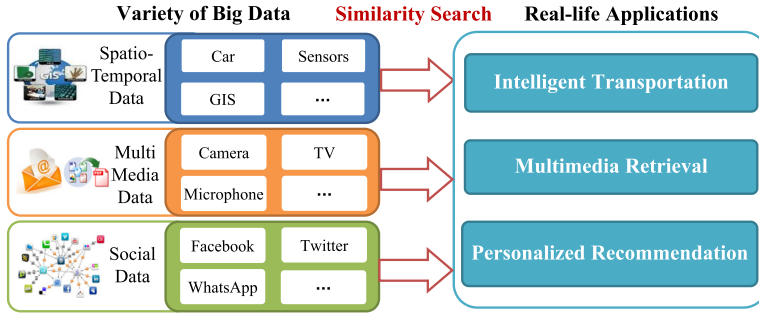


Fig. 1. Applications of similarity search on variety of big data.

related to index construction; and (iv) we provide empirical comparisons of the similarity search performance achieved by metric indexes.

A key strength of metric space solutions is that they are applicable to a broad range of data types found in real-world applications. While we can design specific indexes for different data types (R-trees [16] for low-dimensional vectors, q-gram inverted indexes for strings [154], HI-tree [93] for documents, etc.), metric indexing offers a wholesale solution that spans a broad range of data, the key requirement simply being that a distance function is provided that satisfies the triangle inequality. Since the early 1980's, major DBMS vendors have followed a "one-size-fits-all" approach, due in part to the low maintenance cost and ease of use by customers. However, in today's setting, the "one-size-fits-all" approach is increasingly being abandoned [131, 132] due to a number of reasons such as the availability of independent DB engines and better optimization techniques. Furthermore, some database vendors are developing multi-model database systems (notably Azure Cosmos DB developed by Microsoft in 2017 and GeminiDB developed by Huawei in 2019) that aim to support a range of models (e.g., tables, graphs, and documents) within a single system. Such systems easily become very complex. Metric indexing offers an approach to reduce this complexity.

The rest of the article is organized as below. Section 2 presents the basic concepts of metric indexes. Section 3 provides detailed categorizations of metric indexes. Section 4 summarizes the techniques used by metric indexes. Section 5 describes each metric index in turn. Experimental findings are reported in Section 6. Finally, Section 7 concludes and provides future research directions.

2 BASIC CONCEPTS

We provide the basic definitions of metric spaces and similarity search, and we introduce and categorize the metric indexes. Table 1 summarizes notations used throughout this article.

2.1 Metric Space

A metric space is a two-tuple (M, d) , in which M is an object domain and d is a distance function that measures the "similarity" between objects in M . In particular, the distance function d has four properties: (i) *Symmetry*: $d(q, o) = d(o, q)$; (ii) *non-negativity*: $d(q, o) \geq 0$; (iii) *identity*: $d(q, o) = 0$ iff $q = o$; and (iv) *triangle inequality*: $d(q, o) \leq d(q, p) + d(p, o)$.

Any type of data combined with a distance function that satisfies the above four properties constitutes a metric space. Hence, the notation of metric space is very general. A typical example of a metric space is a vector space associated with the L_p -norm ($p \geq 1$). Another example of a metric space is a set of strings along with the edit distance. Note that vectors are easy and

Table 1. Notations Used Throughout This Article

Symbol	Description
O, C, P	The sets of data, centers, and pivots
S	The sample set of O
o, c, p	An object, center, and pivot
M	An object domain
$d(o, p)$	The distance between objects o and p
n, n_s	The cardinality of dataset O and the cardinality of the sample set S
s	The storage size of an object
m	The tree arity or the capacity of a tree node
l, l_c	The number of pivots and the number of candidate pivots
g	The number of pivots in a pivot group
n_d	The number of values for a discrete $d()$ or the maximum distance of a continuous $d()$
$MRQ(q, r)$	The metric range query with query object q and search radius r
$MkNNQ(q, k)$	The k nearest neighbor query with query object q and result cardinality k
ND_k	The distance from q to its nearest neighbor
$N_i, AN(o)$	A tree node, the accessory nodes of o
MBB	The minimum bounding box of a node
$\phi(o)$	The mapped vector $\langle d(o, p_1), d(o, p_2), \dots, d(o, p_l) \rangle$ using a pivot set $P = \{p_1, \dots, p_l\}$
$SR(q)$	The mapped search region using P
$R_i, A(p_i)$	A particular partition obtained via different partitioning techniques, a partition corresponds to pivot p_i
d_{med}	The medium distance used to obtain two partitions of equal size
δ, ρ, α	Distance thresholds
μ, σ^2	The mean and variance of distances between data objects
μ_{p_i}	The expected value of $d(o, p_i) (o \in A(p_i))$
$N(c)$	The so-called nearest neighbors for a node c defined in SAT

efficient to process. Thus, vector spaces (including Euclidean spaces) often find use in real-world applications. Vector spaces are also used by the different solutions for metric space. For example, pivot mapping is used to represent objects as vectors of their distances to pivots (cf. Section 4.2). Unlike embedding techniques (e.g., word2vec [41]) that map other data types to vectors, pivot-based solutions can ensure accurate (and thus explainable) results when given a specific metric. This may be important in applications where liability is an issue. For example, although genes can be embedded as vectors [133], embedding does not enable accurate and explainable results, which are important in discovering the evolutionary relationship between species [147], genome databases, and so on.

Various indexes exist that support particular metric spaces, e.g., the R-tree [16], KD-tree [17], and TV-tree [82] for low-dimensional vector spaces, where properties (e.g., the dimension information of vector spaces) of the particular metric space are utilized to accelerate the search. However, such specific properties are not available in general metric spaces, and thus, they cannot be used for search space pruning in general metric spaces. For example, the specific properties of vectors cannot be applied to strings. By using metric space properties, indexing solutions can be developed for processing a wide variety of data. Nonetheless, due to the generality of metric spaces, we can only use the four distance properties discussed above for pruning and hence accelerate search. In Section 4, we detail the techniques that can be used in general metric spaces.

Intrinsic Dimensionality of Metric Space. The dimensionality is an important aspect of vector space data. The higher the dimensionality, the worse the search performance will generally be. However, metric spaces are not limited to vector spaces, but also include other data types such as strings and sets. Instead of dimensionality, we can use the more general notion of **intrinsic dimensionality (ID)** that applies to any data type in metric spaces. As discussed in previous studies [18, 35, 47, 86, 112, 122, 145, 151], ID affects metric space search performance just like dimensionality affects vector space search performance. Many methods exist to compute ID. They can be classified into local and global methods. The local methods [5, 70–73] provide different dimensionality estimates for each data object or each subset of a dataset, while the global methods [99]

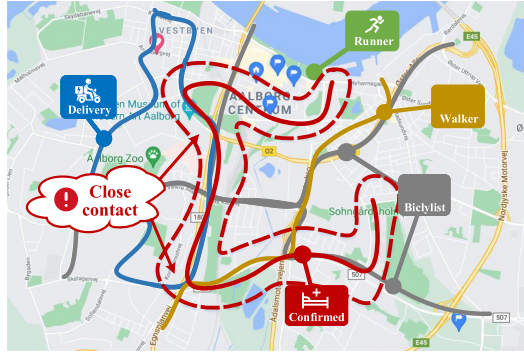


Fig. 2. Illustration of exact similarity search for COVID-19 infected person detection.

provide a dimensionality estimate for an entire dataset. A simple global estimate is introduced in a previous survey [35], i.e., ID is calculated as $\mu^2/2\sigma^2$, where μ and σ^2 are the mean and variance of the distances between data objects. This simple notion of ID is found to be verified effective in an experimental study [99]. Hence, we use this simple but effective definition of ID of datasets in our experiments.

2.2 Similarity Queries in Metric Spaces

We proceed to define similarity search in metric spaces, including metric range query and metric k nearest neighbor query. We focus on similarity queries because of their importance and widespread use, and because they are suitable for studying efficiency and effectiveness of the designed indexes.

Definition 2.1 (Metric Range Query). Given an object set O , a query object q , and a search radius r in a metric space, a metric range query returns the objects in O that are within distance r of q , i.e., $MRQ(q, r) = \{o | o \in O \wedge d(q, o) \leq r\}$.

Definition 2.2 (Metric K Nearest Neighbor Query). Given an object set O , a query object q , and an integer k in a metric space, a metric k nearest neighbor query finds k objects in O that are most similar to q , i.e., $MkNNQ(q, k) = \{S | S \subseteq O \wedge |S| = k \wedge \forall s \in S, \forall o \in O - S (d(q, s) \leq d(q, o))\}$.

Consider the DNA set $O = \{\text{"ATAGCTTCA"}, \text{"AATCTGA"}, \text{"AATCTGT"}, \text{"AAAACGG"}, \text{"CATCTGT"}\}$, where edit distance is employed. An example metric range query finds the DNAs from O within edit distances no larger than 2 of the query DNA "CAATCTGT", i.e., $MRQ(\text{"CAATCTGT"}, 2) = \{\text{"AATCTGA"}, \text{"AATCTGT"}, \text{"CATCTGT"}\}$. An example metric k (=2) nearest neighbor query finds the 2 DNAs from O closest to the query DNA "CAATCTGT", i.e., $MkNNQ(\text{"CAATCTGT"}, 2) = \{\text{"AATCTGT"}, \text{"CATCTGT"}\}$.

There are two kinds of "approximate" in similarity search. First, the user may request similar results, i.e., results that are approximately equal to a query object. Second, to achieve higher efficiency, a solution may compute a result that approximates the requested result. The survey covers the computation of approximate queries, but it does not cover solutions that compute results that approximate requested results. Hence, exact similarity search denotes the exact computation of both exact results and approximate results.

- (i) In terms of exact results, we provide a motivating example in public safety applications, where accurate (and explainable) results are required. For example, Figure 2 shows a set of trajectories of moving objects (bicyclists, pedestrians, and runners) and a query trajectory of a COVID-19 infected person (the red curve). In this example, obtaining exact results is desirable to stop the spread of COVID-19. The example uses the L_2 -norm. This similarity

notion—which is also called the L_2 metric—measures the distance between two points in two-dimensional Euclidean space and is used widely in many contexts [151].

- (ii) In terms of approximate results, exact similarity search is important in a wide range of applications such as data cleaning [7], near-duplicate detection [79, 114], and bioinformatics [114].

Strategies for Answering MkNNQ using MRQ. An MkNNQ can be answered by an MRQ, if the distance from q to its k th nearest neighbor, denoted as ND_k , is known. However, ND_k is not known when a query is issued. Three typical methods exist for computing MkNNQ [21, 69].

- (i) **Strategy 1:** MRQs with incremental growing search radius. Specifically, an MRQ with a small radius is performed first, and then the radius is increased gradually until k nearest neighbors are found. Although this strategy tries to avoid visiting objects already verified, it traverses the metric index multiple times, resulting in high cost.
- (ii) **Strategy 2:** Setting the search radius to infinity, and verifying the objects in order while tightening the radius using verification. To be described in Sections 3 and 5, MkNNQ processing usually adopts this second strategy due to its better performance.
- (iii) **Strategy 3:** Using the candidates to calculate the current ND_k , and then performing an MRQ using the current ND_k as the radius to refine the result. The performance of this strategy relies on the quality of the initial candidates.

Performance Metrics for Similarity Search in Metric Spaces. When evaluating the performance of a similarity query, we use running time (i.e., response time) as the performance metric. As metric spaces are general and support a broad range of data types, the associated distance functions also range from being simple (e.g., the L_p -norm for low-dimensional vectors and the hamming distance) to being complex (e.g., edit distance, the L_p -norm for high-dimensional vectors, shortest-path distance, and earth mover's distance [118]). Existing metric indexes and search algorithms aim to reduce the number of distance computations to improve efficiency. Acknowledging that the distance function may be simple in some applications, indicating that distance computation is not the dominant cost, the survey provides different recommendations for simple versus complex distance functions (cf. Section 7). Since external indexes are stored on disk, the IO time can be estimated as the number of disk **page accesses (PA)** performed during search. In contrast, metric indexes stored in main memory incur no IO time. To summarize, in order to evaluate the performance of similarity search in metric spaces, we utilize three metrics: (i) the running time, (ii) the number of distance computations (*compdists*), and (iii) the number of *PA*.

3 DETAILED CATEGORIZATION OF METRIC INDEXES

We first provide the categorization of metric indexes for exact similarity search, and then cover the detailed categorizations for compact-partitioning based methods and pivot-based methods.

3.1 Categorization of Metric Indexes for Exact Metric Similarity Queries

A rich set of indexes have been proposed that aim to support efficient metric similarity queries. Table 2 provides an overview of existing metric indexes that support exact similarity search. They can be classified into two broad categories: compact-partitioning techniques (termed CP-Indexes) and pivot-based techniques (termed P-Indexes). In addition, hybrid indexes combine compact-partitioning and pivot-based methods (termed Hybrid). In the table, we provide the space and the time cost for index construction. The space and time complexities of most in-memory metric indexes and some secondary-memory metric indexes (such as the M-tree and LC) are already provided in previous surveys or in the original papers, but the time and space complexity results for other in-memory indexes (such as the BU-tree, HC, and EPT^(*)) and most secondary-memory

Table 2. Metric Indexes for Exact Similarity Search

Index	Category	Space Cost	Construction Cost
BST [78], MBST [106]	CP-Index	$O(ns)$	$\Omega(n \log_2 n)$
VT [53, 54]	CP-Index	$O(ns)$	$\Omega(n \log_3 n)$
BU-Tree [83]	CP-Index	$O(ns + n^2)$	$O(n^3)$
GHT [139]	CP-Index	$O(ns)$	$\Omega(n \log_2 n)$
GNAT [23], EGNAT [87, 105]	Hybrid	$O(ns + nm)$	$\Omega(nm \log_m n)$
SAT [32, 97, 98]	CP-Index	$O(ns)$	$O(n \log^2 n / \log \log n)$
DSAT [100–104]	CP-Index	$O(ns)$	$O(mn \log_m n)$
DSACLT [15, 24]	CP-Index	$O(ns)$	$O(mn \log_m n / k)$
kNNG [110]	CP-Index	$O(ns + nk)$	$O(n^2)$
M-tree [42, 46, 128]	CP-Index	$O(ns + ns/m)$	$O(n(m..m^2) \log_m n)$
PM-tree [130]	Hybrid	$O(n(s + l) + n(s + l)/m + ls)$	$O(n(m..m^2) \log_m n + nl \log_m n)$
LC [33, 34], DLC [104]	CP-Index	$O(ns)$	$O(n^2/m)$
HC [62, 63]	CP-Index	$O(ns)$	$O(n \log_2 n/m)$
D-index [55, 56, 150]	Hybrid	$O(ns + nl + ls)$	$O(nl)$
MBT ⁺ -Tree [75]	CP-Index	$O((n + n/m)(s + \log_2 n/m + \log_2 n_d))$	$O(n \log_2 n/m + nm \log_m n)$
AESA [119], ROAESA [144], iAESa [58, 59]	P-Index	$O(ns + n^2)$	$O(n^2)$
LAESA [91]	P-Index	$O(ns + ls + nl)$	$O(nl)$
TLAESA [90, 134]	Hybrid	$O(ns + ls + nl)$	$\Omega(n \log_2 n + nl)$
EPT [120]	P-Index	$O(ns + lgs + nl)$	$O(nlg)$
EPT* [39]	P-Index	$O(ns + l_c s + nl)$	$O(nl l_c n_s)$
CPT [94]	P-Index	$O(ns + ns/m + ls + nl)$	$O(n(m..m^2) \log_m n + nl)$
BKT [25], FQT [14]	P-Index	$O(ns + ln_d)$	$O(nl)$
FHQT [13], FQA [35]	P-Index	$O(ns + nl)$	$O(nl)$
VPT [138, 139, 149], DVPT [64]	P-Index	$O(ns)$	$O(n \log_2 n)$
MVPT [20, 21]	P-Index	$O(ns)$	$O(n \log_m n)$
Omni-family [22, 136]	P-Index	$O(ns + nl + nl/m + ls)$	$O(nml \log_m n)$
SPB-tree [36, 37]	P-Index	$O(ns + n + n/m + ls)$	$O(n(l^2..l^2) + n(m + l) \log_m n)$
M-index [107]	Hybrid	$O(ns + nl + n + n/m + ls)$	$\Omega(nl \log_l n/m) + O(nm \log_m n)$
M-index* [39]	Hybrid	$O(ns + nl + n + n/m + nl/m + ls)$	$\Omega(nl \log_l n/m) + O(nm \log_m n)$

(Construction Cost Includes the Costs of Distance Computations and Other Operations, and Refers to the Following Parameters: The Dataset Cardinality n , the Tree Arity or Capacity m of a Tree Node, the Number l of Pivots, the Cardinality n_s of a Sample Set, the Number l_c of Candidate Pivots, and the Number k of Nearest Neighbors).

metric indexes are not provided in the literature. Some indexes (e.g., the BST family, the GHT family, TLAESA, and the M-index) are unbalanced trees. For these, we assume that the tree structure is balanced to obtain the optimal (i.e., lower bound) construction cost $\Omega(\cdot)$. We omit the similarity search complexity for the indexes, because it depends on the pruning ability that depends on the data distribution. As stated in Section 2.2, we instead report on empirical studies that use three performance metrics to quantify the similarity search performance. In Table 2, different background colors are used to distinguish the families of indexes. For instance, BST, MBST, VT, and the BU-tree belong to the BST family, while GHT, GANT, and EGNAT belong to the GHT family.

Compact-partitioning based Methods. Methods in this category divide the space as compactly as possible, and try to prune unqualified partitions during search. The **Bisector Tree (BST)** [78] is a binary tree that uses a center with a covering radius to represent a partition. Many variants of the BST, including the **Monotonous BST (MBST)** [106], the **Voronoi Tree (VT)** [53, 54], and the **Bottom-Up Tree (BU-Tree)** [83], are developed to improve the efficiency of the BST. The **Generalized Hyperplane Tree (GHT)** [139] is similar to the BST, but does not store covering radius. The **Spatial Approximation Tree (SAT)** [32, 97, 98] is based on Voronoi diagrams, and attempts to approximate the structure of a Delaunay graph. Dynamic and secondary memory extensions of SAT includes DSAT [100–104] and DSACLT [15, 24]. In addition, **k-Nearest-Neighbor Graph (kNNG)** [110] is another popular graph structure. Next, the M-tree [42, 46, 128] is a height-balanced tree optimized for secondary memory. It is the first dynamic index that supports insertion and deletion. Several variants of M-trees have been presented, including the MM-tree [113], Slim-tree [129, 135], M⁺-tree [155], BM⁺-tree [156], and CM-tree [10], which use

different split functions to reduce the overlap among nodes; the M^X -tree [77] and Onion-tree [29, 30] that aim to reduce the tree construction cost; the DBM-tree [142, 143] that allows a controlled imbalance to better accommodate the dataset density variations; the Antipole Tree [28] that aims to minimize the number of clusters; and the BP-tree [2], M^* -tree [126], DF-tree [137], PM-tree [130], and PM^* -tree [126] that combine multiple local or global pivots to further improve the pruning ability of the M-tree. More recently, variants $M^\#$ -tree and $PM^\#$ -tree [116] are designed to avoid duplications of data. The **List of Clusters (LC)** index [33, 34] employs a LC to trade construction time for query time. Its dynamic version is called **Dynamic LC (DLC)** [104]. The construction efficiency of LC can be improved by constructing multiple layers (resulting in HC [62, 63]) or by using cluster reduction [9]. Next, the Metric B^+ -tree (MB^+ -tree) [75] uses relaxed generalized partitioning or hash partitioning to recursively partition the dataset and build a binary tree, while each leaf node denotes a cluster. In particular, each object can be represented as a fixed-length bit string after partitioning and then can be indexed by a B^+ -tree.

Pivot-based Methods. Methods in this category store pre-computed distances from every object in the database to a set of so-called pivots and then utilize these distances to prune unqualified objects during search. The **Approximating Eliminating Search Algorithm (AESA)** [119] utilizes a pivot table to preserve the distances from each object to other objects. To improve search efficiency, the **Reduced-Overhead AESA (ROAESA)** [144] and iAESA [58, 59] adopt the same data structure as AESA, but they sort the pre-computed distances during the search. To reduce the storage of AESA, the **Linear AESA (LAESA)** [91] only keeps the distances from every object to selected pivots. Unlike LAESA that uses a single set of pivots, Extreme Pivot Table $EPT^{(*)}$ [39, 120] employs several sets of pivots. Next, the **Clustered Pivot-table (CPT)** [94] clusters the pre-computed distances to improve query efficiency. The **Burkhard-Keller Tree (BKT)** [25] is designed for discrete distance functions. In contrast to BKT, where pivots at individual levels are different, the **Fixed Queries Tree (FQT)** [14], **Fixed Height FQT (FHQT)** [13], and **Fixed Queries Array (FQA)** [35] use the same pivot for all nodes at the same level of the tree. The **Vantage-Point Tree (VPT)** [138, 139, 149] is designed for continuous distance functions and has been extended to a dynamic structure DVPT [64], and generalized to an m -ary tree yielding the MVPT [20, 21]. The Omni-family [22, 136] employs selected pivots together with existing structures (e.g., the R-tree) to index pre-computed distances. The Space-filling curve and Pivot-based B^+ -tree (SPB-tree) [36, 37] utilizes a space-filling curve to map pre-computed distances to integers, which are then indexed by the B^+ -tree. Note that, although SFC used in the SPB-tree can well cluster the data, SPB-tree is not classified to hybrid methods in this survey, as it does not use the partitioning techniques summarized in Section 4.1.

Hybrid Methods. These methods combine compact partitioning with the use of pivots. The **Geometric Near-Neighbor Access Tree (GNAT)** [23] is an m -way generalization of GHT that utilizes the generalized hyperplane partition method to partition the dataset and also uses cut-regions [84] defined by pivots to accelerate similarity search. A dynamic variant, the **Evolutionary Geometric Near-neighbor Access Tree (EGNAT)**, has also been proposed [87, 105]. By combining generalized partitioning and ball partitioning, the **Tree LAESA (TLAESA)** [90, 134] extends LAESA and organizes the data in a tree. The D-index [55, 56, 150] combines hash partitioning and the pivot mapping. It is a multi-level structure which hashes objects to buckets that are search-separable. The PM-tree [130] also uses cut-regions defined by pivots to accelerate similarity queries on the M-tree. The $M\text{-index}^{(*)}$ [39, 107] generalizes the iDistance [76] technique for general metric spaces, which compacts the objects by using pre-computed distances to their closest pivots.

Table 3. Compact-partitioning Based Metric Indexes

Partitioning Technique	Index	Storage	Scalability
Generalized hyperplane partitioning	GHT, GNAT	Main-memory	Static
	EGNAT	Secondary-memory	Dynamic
	kNNG	Main-memory	Dynamic
	M-index ^(*)	Secondary-memory	Dynamic
Ball partitioning	M-tree and PM-tree	Secondary-memory	Dynamic
	LC, DLC, HC	Secondary-memory	Dynamic
Hash partitioning	D-index	Secondary-memory	Dynamic
	MB ⁺ -tree	Secondary-memory	Dynamic
Hybrid partitioning	BST, MBST, VT, BU-tree	Main-memory	Static
	SAT	Main-memory	Static
	DSAT, DSACLT	Secondary-memory	Dynamic
	TLAESA	Main-memory	Static

Other Metric Indexes. This survey cover metric indexes that support exact similarity search; thus, indexes that support other metric search are omitted. For example, to answer similarity joins, the eD-index [57] which extends the D-index is designed. To answer probabilistic range query, the UP-Index [6] and UPB-tree/UPB-forest [38, 40] are developed to index uncertain metric data. To support indexing multiple metric spaces, the M²-tree [43], M³-tree [27], and Reference R*/RR*-tree [61] are proposed. When the ID is high, exact similarity search rarely outperforms sequential scan [112, 146]. In such cases, approximate similarity search (i.e., where high-quality approximate answers instead of exact answers are returned) may be attractive. In particular, approximate similarity search is able to offer increased efficiency over exact similarity search, which is beneficial in cases where fast search is needed and where the metric is too expensive, the dataset size is extremely large, and so on. Many metric indexes are proposed that aim to support approximate similarity search, including approximate M-tree variants [44, 80, 127, 152, 153], hash based methods [11, 12, 89], permutation-based indexes [4, 31, 60, 92, 95, 140], and kNN graph based methods [85, 109, 123, 124], to name but a few. The P-Shere tree [65] is built using a sample of query objects. The DAHC-tree [3] is optimized according to the global data distribution for high-dimension space. In addition, metric indexes can use other techniques (e.g., short term memories [117], bit operations [50], regrouping [121], parallel computing [108], and cost-model-based distance distribution [45]) to improve query efficiency. Different from an individual metric index, an index framework that combines different metric indexes is also available [88].

3.2 Categorization of Compact-Partitioning based Metric Indexes

Compact partitioning methods can be divided into four categories according to the partitioning technique used: *generalized hyperplane partitioning based indexes*, *ball partitioning based indexes*, *hash partitioning based indexes*, and *hybrid partitioning based indexes* (see Table 3). Note that, hybrid methods that use partitioning techniques are also discussed here.

Indexes in the first category use the generalized hyperplane partitioning (as stated in Definition 4.2) to partition the data. Indexes in the second category use ball partitioning (as defined in Definition 4.1) to partition the data. Indexes in the third category utilize the hash partitioning (as defined in Definition 4.3) to organize the data. Indexes in the fourth category combine ball partitioning and hyperplane partitioning. In particular, they employ the hyperplane partitioning to partition the data, but use ball partitioning for representing each partition.

Table 4. Pivot-based Metric Indexes

Category	Index	Storage	Scalability
Pivot-based tables	AESA, ROAESA, iAESa	Main-memory	Dynamic
	LAESA	Main-memory	Dynamic
	EPT	Main-memory	Dynamic
	CPT	Secondary-memory	Dynamic
Pivot-based trees	BKT	Main-memory	Dynamic
	FQT, FHQT, FQA	Main-memory	Dynamic
	TLAESA	Main-memory	static
	GNAT	Main-memory	Static
	VPT, MVPT	Main-memory	Static
	DVPT	Main-memory	Dynamic
Pivot-based secondary-memory indexes	PM-tree	Secondary-memory	Dynamic
	EGNAT	Secondary-memory	Dynamic
	D-index	Secondary-memory	Dynamic
	Omni-family	Secondary-memory	Dynamic
	M-index ^(*)	Secondary-memory	Dynamic
	SPB-tree	Secondary-memory	Dynamic

3.3 Categorization Pivot-based Metric Indexes

Pivot-based methods can be classified into three categories, i.e., *pivot-based tables*, *pivot-based trees*, and *pivot-based secondary-memory indexes*, depending on the structure they use for storing pre-computed distances (see Table 4). Note that, hybrid methods that use pivot mapping techniques are also discussed in this subsection.

Indexes in the first category utilize tables to store pre-computed distances. Indexes in the second category use tree structures to store pre-computed distances. Indexes in the third category utilize an secondary-memory index (e.g., an R-tree or a B⁺-tree) to store pre-computed distances. According to Table 4, although many of the pivot-based methods are dynamic, all the indexes need to be re-built by re-computing all the stored distances when pivots are updated. However, the pivots do not need to be real objects in the dataset; thus, pivots need not necessarily be updated when inserting or deleting data objects. Among all the metric indexes, only two (BKT and FQT) are designed for discrete distance functions that return a finite range of values. Nevertheless, they can be extended to support continuous distance functions.

4 TECHNIQUES FOR METRIC INDEXING AND QUERYING

We summarize the partitioning methods for the compact-partitioning based metric indexes as well as the pivot-based filtering and validation techniques for all the metric indexes.

4.1 Partitioning Methods

Three types of partitioning methods exist, i.e., ball partitioning, generalized hyperplane partitioning, and hash partitioning, as described below.

Definition 4.1 (Ball Partitioning). Given a center c and a radius r , the set of objects o in ball partition R_i is defined as $\{o | o \in O \wedge d(o, c) \leq r\}$.

In the example illustrated in Figure 3(a), given the center o_7 and radius $d(o_7, o_6)$, we can obtain the ball partition $R_i = \{o_6, o_7, o_8\}$. The ball radius can be set to 2^i ($-\infty \leq i \leq +\infty$) at each tree level i [19, 74], to contend with the curse of high ID or to contend with the cases when distance functions do not satisfy the triangle inequality.

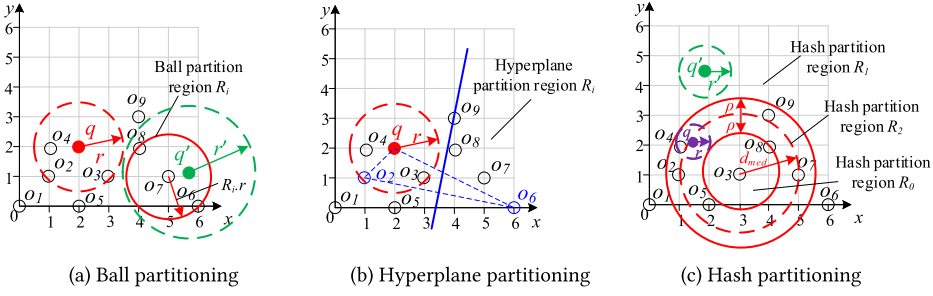


Fig. 3. Illustration of partitioning methods.

Definition 4.2 (Generalized Hyperplane Partitioning). Given a set C of centers, let $c_i \in C$ be the center of partition region R_i . The set of objects o in generalized hyperplane partition R_i is defined as $\{o | o \in O \wedge \forall c_j \neq c_i (d(o, c_i) \leq d(o, c_j))\}$.

In Figure 3(b), given the two centers o_2 and o_6 , we can get two hyperplane partitions $R_1 = \{o_1, o_2, o_3, o_4, o_5\}$ and $R_2 = \{o_6, o_7, o_8, o_9\}$. The generalized hyperplane partitioning can be relaxed by introducing a threshold δ [48, 75], such that partition R_i , obtained by relaxed generalized hyperplane partitioning, is defined as $\{o | o \in O \wedge \forall c_j \neq c_i (d(o, c_i) \leq d(o, c_j) + \delta)\}$. This relaxed method can be used in hyperplane partitioning based indexes (e.g., BST and GNAT).

Definition 4.3 (Hash Partitioning). Given a hash function h , the set of objects o in hash partition R_i , obtained by the hash partitioning, is defined as $\{o | o \in O \wedge h(o) = i\}$.

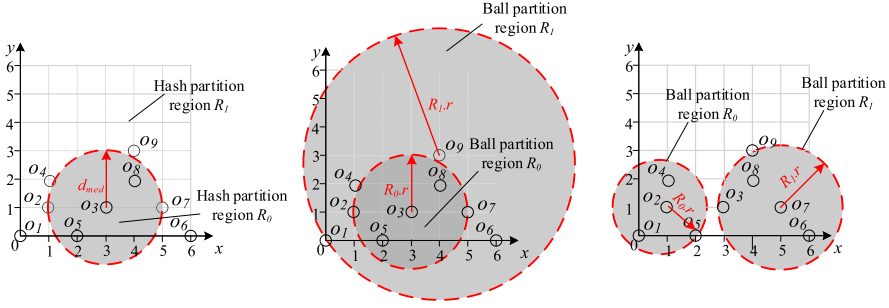
A particular hash function, i.e., the ρ -split function $bps^\rho(c, o)$ uses a center c and the medium distance d_{med} to partition the data into three subsets as defined below. The median distance d_{med} is relative to c and is defined so that the number of objects with distances smaller than d_{med} is the same as the number of objects with distances larger than d_{med} .

$$bps^\rho(c, o) = \begin{cases} 0 & \text{if } d(c, o) \leq d_{med} - \rho \\ 1 & \text{if } d(c, o) > d_{med} + \rho \\ - & \text{otherwise} \end{cases} \quad (1)$$

Here, the result “-” denotes the last partition, the exclusion partition. In Figure 3(c), given a center o_3 , we get three hash partitions: $R_0 = \{o_3\}$, $R_1 = \{o_1, o_6\}$, and $R_2 = \{o_2, o_4, o_5, o_7, o_8, o_9\}$. Here, R_2 is the exclusion partition. The ρ -split function can be generalized to a set of centers. Given a set of m centers C , the objects can be divided into $2^m + 1$ partitions. Specifically, $bps^\rho(C, o) = bps^\rho(c_i, o) \times \sum_{x=1}^i 2^{i-1} (c_i \in C, 1 \leq i \leq m)$ if $bps^\rho(c_i, o) = 0$ or 1; otherwise, $bps^\rho(C, o) = 2^m$.

The hash partitioning used in metric spaces is mainly extended from ball partitioning (e.g., [56]) and hyperplane partitioning (e.g., [89]). This is natural because ball and hyperplane partitioning make it easy to ensure the correctness of similarity search, and thus, to support exact similarity search. However, hash functions can also be designed specifically for approximate similarity search (e.g., [125]), which is a promising future direction for designing metric indexes.

Given m centers, ball (or hyperplane) partitioning divides the data space into m subspaces, while ρ -split hash partitioning divides the data space into $2^m + 1$ subspaces. Although ρ -split hash partitioning can be regarded as ball partitioning, it is not a good split for ball partitioning. Figure 4 illustrates ball and hash partitioning. In Figure 4(a), the hash function $bps^0(o_3, o)$ ($\rho = 0$) divides the space into two disjoint subspaces, including hash partition region R_0 (the grey circular region) and hash partition region R_1 (the white region). The hash partitioning can be regarded as a ball partitioning with centers o_3 and o_9 , as shown in Figure 4(b). However, the split is not good, as the



(a) Hash partitioning (center o_3) (b) Ball partitioning (centers o_3 and o_9) (c) Ball partitioning (centers o_2 and o_7)

Fig. 4. Comparison of ball partitioning and hash partitioning.

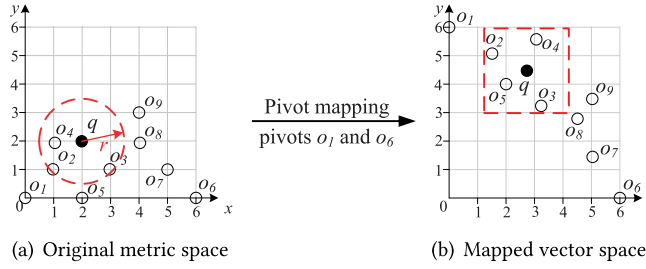


Fig. 5. Pivot mapping.

overlap between ball regions R_0 and R_1 is large, resulting in poor pruning capabilities. If we instead use centers o_2 and o_7 for ball partitioning, as depicted in Figure 4(c), we can obtain a better split. Figure 4 illustrates two differences between hash and ball partitioning: (i) Their representations are different. Metric indexes that use ball partitioning to represent each region as a ball, while metric indexes that use ρ -split hash partitioning to only represent some of regions as balls. (ii) How to select the centers is different. Existing studies [26, 86, 111, 141, 148] focus mostly on how to select high-quality pivots for pivot-based metric indexes, and only few studies [32, 46] consider how to select high-quality centers for compact-partitioning based metric indexes.

4.2 Pivot Mapping

By using a set of pivots, the objects in a metric space can be mapped to data points in a vector space. In particular, given a pivot set $P = \{p_1, p_2, \dots, p_l\}$, a metric space (M, d) can be mapped to a vector space (R^l, L_∞) . Specifically, an object q in the metric space is mapped to a point $\phi(q) = \langle d(q, p_1), d(q, p_2), \dots, d(q, p_l) \rangle$ in the vector space.

Consider the example in Figure 5, where the L_2 -norm is used as the distance function. If $P = \{o_1, o_6\}$, the object set in the original metric space can be mapped to the data points in a two-dimensional vector space, in which the x-axis denotes $d(o_i, o_1)$ and the y-axis represents $d(o_i, o_6)$ for any object o_i . For example, object o_5 is mapped to point $\langle 2, 4 \rangle$.

4.3 Pivot-based Filtering and Validation

The triangle inequality is the only property that can be used to reduce the search space in general metric spaces. Below, we summarize seven filtering and validation lemmas. In the case of both compact-partitioning and pivot-based metric indexes, centers and pivots are combined with the

triangle inequality for pruning and validation. A center used for compact partitioning methods can be regarded as a pivot of pivot-based methods. First, pivot-based filtering [36] based on the pivot mapping is utilized to avoid unnecessary distance computations.

LEMMA 4.1 (PIVOT FILTERING). *Given a set $P = \{p_1, p_2, \dots, p_l\}$ of pivots, a query object q , and a search radius r , let $SR(q) = \langle [d(q, p_1) - r, d(q, p_1) + r], \dots, [d(q, p_l) - r, d(q, p_l) + r] \rangle$ be a mapped search region. If a mapped $\phi(o) = \langle d(o, p_1), d(o, p_2), \dots, d(o, p_l) \rangle$ locates outside $SR(q)$, the original object o can be pruned safely.*

PROOF 4.1. Assume, to the contrary, that an object o exists that satisfies $d(q, o) \leq r$, but $\phi(o) \notin SR(q)$ (i.e., $\exists p_i \in P (d(o, p_i) > d(q, p_i) + r \vee d(o, p_i) < d(q, p_i) - r)$). By the triangle inequality, $d(q, o) \geq |d(q, p_i) - d(o, p_i)| > r$, which contradicts our assumption, and completes the proof. $\square$

Since the pre-computed distances in $\phi(o)$ are stored together with object o in a metric index, we can avoid distance computations involving object o if $\phi(o) \notin SR(q)$, based on Lemma 4.1. Consider the example in Figure 5 where the dashed rectangle represents search region $SR(q)$. Here, object o_1 can be pruned as $\phi(o_1) \notin SR(q)$. Also, Lemma 4.1 can be utilized to prune an entire region (i.e., a minimum bounding box that contains multiple $\phi(o)$ s) if it does not intersect $SR(q)$. Next, based on ball partitioning, we present a range-pivot filtering technique [151].

LEMMA 4.2 (RANGE-PIVOT FILTERING). *Given a ball partition region R_i with the center $R_i.p$ and radius $R_i.r$, a query object q , and a search radius r , if $d(q, R_i.p) > R_i.r + r$, R_i can be pruned.*

PROOF 4.2. For any object o in R_i , if $d(q, R_i.p) > R_i.r + r$, then $d(q, o) \geq d(q, R_i.p) - d(o, R_i.p) > R_i.r + r - d(o, R_i.p)$ due to the triangle inequality. As $d(o, R_i.p) \leq R_i.r$ by Definition 4.1, we can derive that $d(q, o) > r$. Thus, no object o in R_i can be in the result set, and R_i can be pruned safely. $\square$

Consider Figure 3(a), where the red dashed line denotes the search region, and the solid red circle represents the ball region $R_i = \{o_6, o_7, o_8\}$ with center $R_i.p = o_7$ and radius $R_i.r = d(o_7, o_6)$. As $d(q, R_i.p) > R_i.r + r$, R_i can be pruned due to Lemma 4.2. Note that, for an object o inside ball region R_i , if we record its distance $d(o, R_i.p)$ to the partition center $R_i.p$, Lemma 4.2 can be applied to prune this object by replacing $R_i.r$ with $d(o, R_i.p)$. Hence, Lemma 4.2 can also be used for pruning single objects. Next, based on the generalized hyperplane partitioning, a double-pivot filtering technique [151] exists.

LEMMA 4.3 (DOUBLE-PIVOT FILTERING). *Given pivots p_i and p_j , a query object q , and a search radius r , if $d(q, p_i) - d(q, p_j) > 2r$, R_i can be pruned, as p_i is the corresponding pivot for R_i .*

PROOF 4.3. For every o in R_i , according to the definition of R_i , $d(o, p_i) \leq d(o, p_j)$. Based on the triangle inequality, we have $d(q, p_i) \leq d(o, p_i) + d(q, o)$ and $d(q, p_j) \geq d(o, p_j) - d(q, o)$. Thus, we can derive that $d(q, p_i) - d(q, p_j) \leq d(o, p_i) + d(q, o) - d(o, p_j) + d(q, o) \leq 2d(q, o)$ as $d(o, p_i) \leq d(o, p_j)$. If $d(q, p_i) - d(q, p_j) > 2r$, then $d(q, o) > r$. Therefore, no object $o \in R_i$ can be a result object, and R_i can be pruned. $\square$

Consider Figure 3(b), where o_2 and o_6 are pivots. Since $d(q, o_6) - d(q, o_2) > 2r$, $R_i = \{o_6, o_7, o_8, o_9\}$ can be discarded safely according to Lemma 4.3.

LEMMA 4.4 (EXCLUSIVE FILTERING). *Given a ρ -split function $bps^\rho(c, o)$, a query object q , and a search radius r , if $r \leq \rho$ and $bps^{\rho-r}(c, q) = \text{"-"}$, objects in partitions R_0 and R_1 can be pruned safely; if $bps^{r-\rho}(c, q) = 0$, objects in partition R_1 can be pruned; and if $bps^{r-\rho}(c, q) = 1$, objects in partition R_0 can be pruned.*

PROOF 4.4. If $r \leq \rho$ and $bps^{\rho-r}(c, q) = \text{"-"}$, we can get that $d_{med} - \rho + r < d(q, c) \leq d_{med} + \rho - r$. For objects o in R_0 (i.e., $d(c, o) \leq d_{med} - \rho$), $d(q, o) \geq d(q, c) - d(c, o) > d_{med} - \rho + r - d_{med} + \rho = r$,

and thus, objects in R_0 can be pruned. For objects o in R_1 (i.e., $d(c, o) > d_{med} + \rho$), $d(q, o) \geq d(c, o) - d(c, q) > d_{med} + \rho - d_{med} - \rho + r = r$, and hence, objects in R_1 can be pruned.

If $bps^{r-\rho}(c, q) = 0$, we can get $d(q, c) \leq d_{med} - r + \rho$. For objects in R_1 (i.e., $d(c, o) > d_{med} + \rho$), $d(q, o) \geq d(c, o) - d(q, c) > d_{med} + \rho - d_{med} + r - \rho > r$. Therefore, objects in R_1 can be pruned.

If $bps^{r-\rho}(c, q) = 1$, we can get $d(q, c) > d_{med} + r - \rho$. For objects o in R_0 (i.e., $d(c, o) \leq d_{med} - \rho$), $d(q, o) \geq d(q, c) - d(c, o) > d_{med} + r - \rho - d_{med} + \rho > r$. Thus, objects in R_0 can be pruned. $\square$

Consider Figure 3(c) and assume that the search region $SR(q, r)$ is the dashed purple circle. Partitions R_0 and R_1 can be pruned due to $r \leq \rho$ and $bps^{\rho-r}(c, q) = \text{"-"}.$ Lemma 4.4 uses one center for the ρ -split function to illustrate the exclusive filtering. However, Lemma 4.4 can also be extended to multiple centers.

Lemmas 4.1 through 4.4 are pivot filtering techniques. A distance computation is still needed for verifying each object that cannot be pruned. Consequently, validation techniques are introduced to save unnecessary verifications.

LEMMA 4.5 (PIVOT VALIDATION). *Given a pivot set P , a query object q , and a search radius r , if there exists, for an object o in O , a pivot $p_i \in P$ satisfying $d(o, p_i) \leq r - d(q, p_i)$, o is validated to be a result object.*

PROOF 4.5. Given a query object q , an object o , and a pivot p_i , $d(q, o) \leq d(o, p_i) + d(q, p_i)$ because of the triangle inequality. If $d(o, p_i) \leq r - d(q, p_i)$, then $d(q, o) \leq r - d(q, p_i) + d(q, p_i) = r$. Thus, o is guaranteed to belong to the search region, which completes the proof. $\square$

In Figure 5(b), object o_2 can be validated directly without computing the distance $d(q, o_2)$ due to $d(o_2, o_1) = r - d(q, o_1)$ according to Lemma 4.5.

LEMMA 4.6 (RANGE-PIVOT VALIDATION). *Given a ball partition region R_i with center $R_i.p$ and radius $R_i.r$, a query object q , and a search radius r , if $d(q, R_i.p) \leq r - R_i.r$, objects in R_i are validated as result objects.*

PROOF 4.6. For any object o contained in R_i (i.e., $d(o, R_i.p) \leq R_i.r$), if $d(q, R_i.p) \leq r - R_i.r$, then $d(q, o) \leq d(o, R_i.p) + d(q, R_i.p) \leq r - R_i.r + R_i.r = r$. Hence, o is in the search region according to Definition 2.1, and all objects in R_i are validated as result objects. $\square$

Consider the example shown in Figure 3(a), where the dashed green circle represents the search region, and the solid red circle denotes the ball region $R_i = \{o_6, o_7, o_8\}$ with center $R_i.p = o_7$ and radius $R_i.r = d(o_7, o_6)$. As $d(q, R_i.p) < r - R_i.r$, R_i is validated according to Lemma 4.6.

LEMMA 4.7 (EXCLUSIVE VALIDATION). *Given a ρ -split function $bps^\rho(c, o)$, a query object q , and a search radius r , if $bps^{\rho+r}(c, q) = 0$ (or 1), the query result must be contained in R_0 (or R_1).*

PROOF 4.7. If $bps^{\rho+r}(c, q) = 0$, we can get that $d(q, c) \leq d_{med} - \rho - r$. For any object o outside R_0 (i.e., $d(c, o) > d_{med} - \rho$), $d(q, o) \geq d(o, c) - d(q, c) > d_{med} - \rho - d_{med} + \rho + r = r$, and thus, the query result must be contained in R_0 . If $bps^{\rho+r}(c, q) = 1$, we can get that $d(q, c) > d_{med} + \rho + r$. For any object o outside R_1 (i.e., $d(c, o) \leq d_{med} + \rho$), $d(q, o) \geq d(q, c) - d(c, o) > d_{med} + \rho + r - d_{med} - \rho = r$, and hence, the query result must be contained in R_1 . $\square$

In Figure 3(c), let the dashed green circle be the search region. The search result must be contained in the hash partition region R_1 due to $bps^{\rho+r'}(c, q') = 1$ according to Lemma 4.7. Consequently, we only need to verify the objects in R_1 .

In addition to general pivot filtering principles applicable in all metric spaces, alternative filtering approaches are proposed for exact metric search, such as Ptolemaic filtering [67, 68] and Hilbert exclusion [49, 51]. These alternative filtering approaches use properties in addition to the triangle inequality in order to accelerate search, while the additional properties hold for a series of metrics.

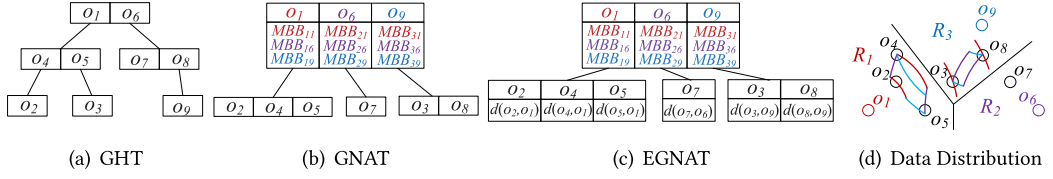


Fig. 6. Examples of GHT and its variants.

5 METRIC INDEXES FOR EXACT SIMILARITY SEARCH

In order to illustrate different designs of the metric indexes, we index the dataset shown in Figure 3 using each of the metric indexes. Sections 5.7–5.14 offer an extended presentation of pivot-based indexes covered in our previous work [39].

5.1 The GHT Family

The GHT [139] is a binary tree built by using the generalized partitioning recursively. In the example of GHT depicted in Figure 6(a), objects o_1 and o_6 in the node denote the centers of two subtrees.

GHT can be generalized to m -ary trees, yielding GNAT [23]. An example of GNAT is depicted in Figure 6(b). When constructing GNAT, m centers c_i ($1 \leq i \leq m$) are selected each time, and objects are assigned to the nearest center. In addition, GNAT stores the minimum bounding box $MBB_{ij} = [\text{mindist}(o, c_j), \text{maxdist}(o, c_j)]$ ($o \in R_i$) of each node with respect to each centers c_j , as shown in Figure 6(d), where red circles denote the MBB w.r.t. center o_1 , the purple circles represent the MBB w.r.t. o_6 , while the blue circle denotes the MBB w.r.t. o_9 .

The EGNAT [87, 105] is a dynamic version of GNAT. An example is shown in Figure 6(c). It supports insertion and deletion, as well as extends GNAT to be an external memory index. In addition, to improve the pruning ability, each entry in the leaf nodes of EGNAT stores the distance from the entry to its parent entry.

Discussion. The storage cost of GHT is $O(ns)$, and its construction cost is $\Omega(n \log_2 n)$, while GNAT has storage cost $O(ns + mn)$ and construction cost $\Omega(mn \log_m n)$, where n denotes the total number of objects, s represents the size of an object, and m denotes the tree arity. In the original study [23], GNAT has $O(m^2 n_{\text{node}})$ MBB storage cost, where n_{node} denotes the number of non-leaf nodes. Here, we use $O(n/m)$ to estimate n_{node} so that $O(mn)$ space is used to store the MBB information. Note that, GHT, GNAT, and EGNAT are unbalance trees, meaning that their worst construction cost is $O(n^2)$. If we assume that the tree structure is balanced, we get optimal construction costs of $\Omega(n \log_2 n)$ for GHT and $\Omega(mn \log_m n)$ for GNAT and EGNAT.

5.2 BST Family

Like GHT, the BST [78] is a binary tree, which is constructed by inserting objects one by one. However, unlike GHT, BST uses a ball (i.e., a center with a radius) to represent each partition/node. To improve efficiency by obtaining a relatively balanced structure, BST can be built recursively in a top-down manner using generalized hyperplane partitioning, yielding BST*. Figure 7 shows an example of BST*, where N_2 is represented by a ball with center o_1 and radius $r_2 = \sqrt{5}$. In contrast, a variant called the BU-tree [83] builds the tree in a bottom-up manner instead of top-down. In BST, a sub node may have a larger radius than its parent node. To avoid this, two variants of BST, called MBST [106] and VT [53], are proposed, where VT nodes have arity 3.

Discussion. The storage costs of BST(*) and MBST are $O(ns)$, and the corresponding construction costs are $\Omega(n \log_2 n)$. The degree of balance depends on the chosen centers and the data

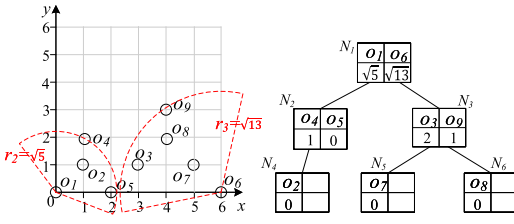


Fig. 7. BST.

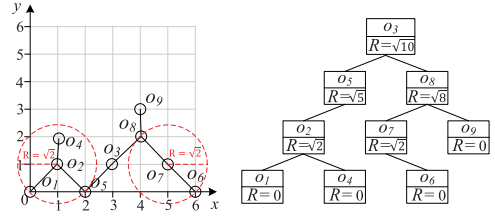


Fig. 8. SAT.

distribution. If the centers used are chosen well, the BST and its variants can be balanced trees. As the VT nodes have arity 3, its construction cost is $\Omega(n \log_3 n)$. However, the BU-tree repeatedly forms a parent node from the two nodes with the minimum distance. Thus, its construction cost is $O(n^3)$.

5.3 SAT Family

In contrast to GNAT, the SAT [97, 98] selects centers inspired by the Delaunay graph. An example is shown in Figure 8. For each node c , SAT chooses a set of so-called nearest neighbors $N(c)$ as the centers of its sub-nodes. More specifically, $N(c)$ does not denote the real nearest neighbors of c , but consists of the objects that are closer to c than to other objects in $N(c)$, i.e., $o \in N(c)$ iff $\forall u \in N(c) - \{o\}, d(o, c) < d(o, u)$. Next, SAT uses the generalized hyperplane partitioning to construct the tree, while each node in SAT also uses the ball representation. As an optimization, the farthest outlier data is selected as the root node instead of a randomly chosen root to maximize the hyperplane separation [32]. In addition, when dividing the dataset in each iteration, the objects are sorted in ascending order of their distances to the parent entry. Another typical index called the k NNG is similar as SAT. The difference is that while SAT uses $N(c)$ as sub-nodes, k NNG uses real k nearest neighbors of c as its sub-nodes.

Dynamic SAT (DSAT) [100–104] extends SAT from being a static in-memory index to being a dynamic secondary-memory index. DSAT is built by inserting objects one by one using generalized hyperplane partitioning, and each object is associated with an insertion timestamp. The sub-nodes are sorted in ascending order of their insertion times. To better cluster each page of DSAT, DSAT with clusters (DSACLT) [15, 24] is proposed. Each node in DSACLT is stored as a single page in secondary memory. In addition, DSACLT stores **k nearest neighbors (k NNs)** of the node center in each node. Furthermore, the distance from each k NN to the node center is stored. Two DSACLT versions exist, DSACLT⁺ and DSACLT*, where DSACLT⁺ is tree structured, while DSACLT* is a list.

Discussion. The construction costs of SAT, DSAT, and DSACLT are $O(n \log^2 n / \log \log n)$, $O(nm \log_m n)$, and $O(nm \log_m n / k)$, respectively, where m is the maximum tree-arity and k denotes the number of nearest neighbors stored in each DSACLT node. The storage costs of SAT, DSAT, and DSACLT are all $O(ns)$. Note that, the construction cost of k NNG is $O(n^2)$ in the worst case and its storage cost is $O(ns + nk)$. Unlike other indexes (e.g., the GHT family) that only use sibling centers for pruning, the SAT family indexes can also use ancestor nodes for pruning, and thus, offer improved pruning capabilities. Due to the dynamic insertions, the pruning of DSAT is weaker than that of SAT. DSACLT stores more pre-computed distances in each node than those of SAT and DSAT, which improves its pruning capabilities over these indexes. However, the height of DSACLT is smaller than that SAT and DSAT, which reduces its pruning capabilities over these. Consequently, the combined pruning enabled by DSACLT depends on the data distribution.

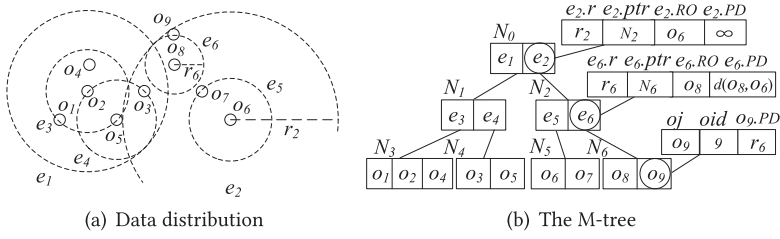


Fig. 9. Example of the M-tree.

5.4 M-tree Family

The M-tree [42, 46, 128] is a dynamic tree that uses ball partitioning. Figure 9 shows an M-tree, where each non-leaf entry e records a center $e.RO$, a covering radius $e.r$, a parent distance $e.PD$ that equals the distance from e to the center of its parent entry, and a pointer to the root of its sub-tree. A leaf entry stores detailed object information with the parent distance.

Many M-tree variants exist (cf. Section 3.1) that aim to improve its construction or query efficiency. Here, we only introduce a representative variant called the PM-tree [130]. The PM-tree combines the pivot mapping and the M-tree, where the M-tree is used to cluster objects and the pivot mapping is utilized to avoid unnecessary distance computations. Hence, unlike the M-tree, each leaf entry of the PM-tree stores the mapped vector (i.e., the pre-computed distances to the pivots) with every object. In each non-leaf entry, the PM-tree stores an MBB that contains all the mapped vectors in its child leaf entries.

Discussion. The M-tree is a balanced tree, and thus, the storage cost of the M-tree is $O(ns + ns/m)$, and its construction cost is $O(nm \log_m n)$ or $O(nm^2 \log_m n)$, depending on the split strategy. The tree-arity m of the M-tree depends on its disk page size to store each node. The PM-tree stores pre-computed distances w.r.t. the pivots in the tree structure. Hence, the construction cost of the PM-tree is $O(n(m..m^2) \log_m n + nl \log_m n)$, and its storage cost is $O(n(s+l) + n(s+l)/m + ls)$, where l denotes the number of pivots, and it needs $O(ls)$ space to store the pivots. The PM-tree costs are relatively high compared with those of the M-tree. Note that, the M-tree and its variants (e.g., the PM-tree) store the data objects in its entries instead of in a separate file; thus, the page/node size varies for different types of data. In particular, for complex objects (e.g., the 282 dimensional vectors used in our experiments), the M-tree family indexes need a large page size.

5.5 LC Family

LC) [33, 34] is a LC, where each cluster is represented by a center and a radius. Each cluster has a corresponding bucket, which contains objects whose distances to the center are not larger than the radius. LC has two versions, i.e., fixed radius and fixed size. Fixed radius means that the cluster radius is fixed, while fixed size indicates that the number of objects in each bucket is fixed. Figure 10 shows an example of LC with fixed size. Two variants DLC [104] and **Hierarchy of Clusters** [62, 63] (HC) are proposed, where DLC is a dynamic version of LC, and HC utilizes a tree structure to store the LC.

Discussion. The construction costs of LC and DLC are $O(n^2/m)$, where m denotes the number of objects in each bucket. The construction cost of LC is very high; however, it can achieve high query efficiency due to its compact clusters. HC is built recursively in a binary way to reduce the construction cost of LC, which yields a construction cost of $O(n \log_2 n/m)$. The storage sizes of LC, DLC, and HC are all $O(ns)$.

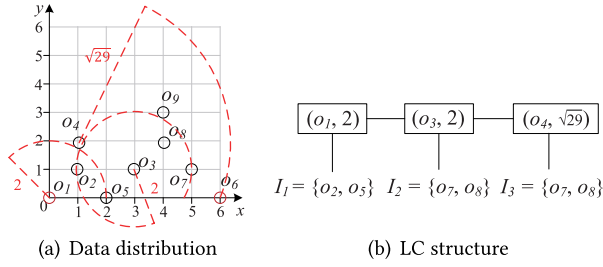


Fig. 10. Example of LC with fixed size.

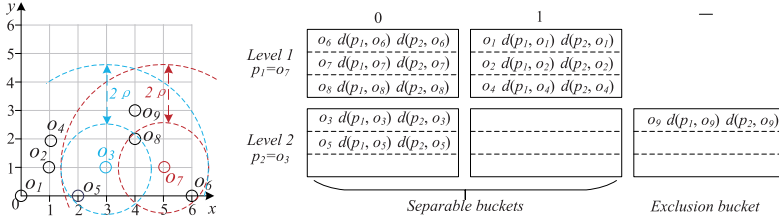
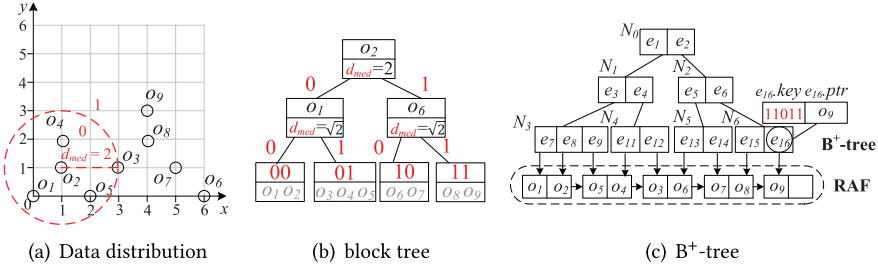


Fig. 11. Example of the D-index.

Fig. 12. Example of the MB⁺-tree using hash partitioning.

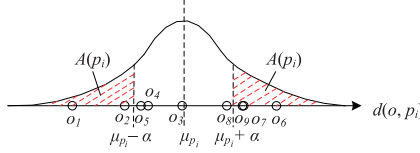
5.6 D-index Family

The D-index [55, 56, 150] combines hash partitioning and the pivot mapping. The basic idea of the D-Index is to create a multi-level structure that uses several ρ -split functions as defined in Definition 4.3, one for each level, to create buckets for storing objects. Here, the ρ -split functions of individual levels use the same ρ . In Figure 11, a ρ -split function based on o_7 is used at level 1, and a ρ -split function based on o_3 is used at level 2. Objects in the exclusion bucket “-” (i.e., o_3, o_5, o_9) at level 1 are candidates to be divided at level 2, and the exclusion bucket of the last level forms the exclusion bucket of the D-index.

The MB⁺-tree [75] divides the dataset into two subsets by using either hash partitioning or generalized hyperplane partitioning recursively. Figure 12 gives an example of the MB⁺-tree using hash partitioning, where ρ is set to 0. The MB⁺-tree includes two parts, i.e., block tree and B⁺-tree. The block tree stores the partition information, where each internal node records the partition center c and the medium distance d_{med} used for a ρ -split function. For each object o , the MB⁺-tree generates a key that is formed by the partition key pk and the distance key dk . Finally, all keys are indexed by the B⁺-tree, and objects are stored in a separate random access file RAF.

Discussion. The construction cost of the D-index is $O(nl)$, as it depends on the number of pivots (i.e., the number of hash functions) used. The storage cost of the D-index is $O(ns + nl + ls)$,

objects	$d(o_i, o_1)$	$d(o_i, o_6)$
o_1	0	6
o_2	$\sqrt{2}$	$\sqrt{26}$
o_3	$\sqrt{10}$	$\sqrt{10}$
o_4	$\sqrt{5}$	$\sqrt{29}$
o_5	2	4
o_6	6	0
o_7	$\sqrt{26}$	$\sqrt{2}$
o_8	$2\sqrt{5}$	$2\sqrt{2}$
o_9	5	$\sqrt{13}$



objects	$(p_1, d(o_i, p_1))$	$(p_2, d(o_i, p_2))$
o_1	$(o_1, 0)$	$(o_9, 5)$
o_2	$(o_1, \sqrt{2})$	$(o_4, 1)$
o_3	$(o_1, \sqrt{10})$	$(o_9, \sqrt{5})$
o_4	$(o_6, \sqrt{29})$	$(o_4, 0)$
o_5	$(o_1, 2)$	$(o_9, \sqrt{13})$
o_6	$(o_6, 0)$	$(o_4, \sqrt{29})$
o_7	$(o_6, \sqrt{2})$	$(o_4, \sqrt{17})$
o_8	$(o_1, 2\sqrt{5})$	$(o_9, 1)$
o_9	$(o_1, 5)$	$(o_9, 0)$

Fig. 13. Example of LAESA.

Fig. 14. Illustration of $A(p_i)$.

Fig. 15. Example of EPT.

where $O(ns)$ is the cost to store the dataset, $O(nl)$ is cost to store the pre-computed distances w.r.t. the pivots, and $O(ls)$ is the cost to store the pivots used for pivot mapping and hash partitioning. The construction cost of the MB⁺-tree is $O(n \log_2 n/m + nm \log_m n)$, where it needs $O(n \log_2 n/m)$ to build the block tree, and $O(nm \log_m n)$ to build the B⁺-tree, where m denotes the number of entries in both the block tree leaf node and the B⁺-tree node. The storage cost of the MB⁺-tree is $O((n + n/m)(s + \log_2 n/m + \log_2 n_d))$, in which n_d represents the maximum distance value of $d()$. This is because, the length of a partition key is $O(\log_2 n/m)$, while the length of a distance key is $O(\log_2 n_d)$. It needs $O((n + n/m)(\log_2 n/m + \log_2 n_d))$ to store the B⁺-tree, $O(ns)$ to store the RAF, and $O((s + \log_2 n/m)n/m)$ to store the block tree.

5.7 AESA Family

AESA [119] uses a table to store the distances from every object to other objects. If n is the cardinality of a dataset, the storage cost of AESA is $O(n^2)$, which is high for large datasets. As an example, if $n = 100,000$, the storage cost is 80G, which renders AESA a theoretical metric index. To further improve the query efficiency, two variants ROAESA [144] and iAESA [144] are proposed, where the pre-computed distances are visited in a particular order during the search. To reduce the storage cost of AESA, LAESA [91] is developed. LAESA only stores the distances from each object to the pivots in a pivot set P . Figure 13 shows an example of LAESA when using $P = \{o_1, o_6\}$. In addition, objects in LAESA can be organized in a binary tree TLAESA [90, 134] instead of a table.

Discussion. The construction costs of AESA, ROAESA, and iAESA are all $O(n^2)$, while the storage costs of AESA, ROAESA, and iAESA are all $O(ns + n^2)$, as they need $O(ns)$ to store the data and $O(n^2)$ to store the pre-computed distances. To reduce the cost of AESA, the construction cost of LAESA is $O(nl)$, and its storage cost is $O(ns + nl + ls)$, as LAESA needs $O(ns)$ to store the data, $O(nl)$ to store the pre-computed distances, and $O(ls)$ to store pivots. Since TLAESA is a tree structure, the construction cost of TLAESA is $O(n \log_2 n + nl)$, but its storage cost is the same as LAESA. For similarity search processing, although LAESA utilizes Lemma 4.1 to avoid certain unnecessary distance computations, it still needs to scan the full table to find the result set, incurring additional scanning cost.

5.8 EPT

Unlike LAESA that utilizes the same pivots for each object, **Extreme Pivot Table (EPT)** [120], as shown in Figure 15, selects different pivots for different objects in order to achieve better search performance. EPT consists of a set of pivot groups. Each group G contains g pivots $p_i (1 \leq i \leq g)$, according to which the entire dataset O is partitioned into g parts $A(p_i)$, such that $A(p_i) \cap A(p_j) = \emptyset$ ($i \neq j$) and $\cup_{p_i \in G} A(p_i) = O$. An object o belongs to $A(p_i)$ iff $|d(o, p_i) - \mu_{p_i}| \geq \alpha$, where μ_{p_i} is the expected value of $d(o, p_i)$. For instance, in Figure 14, $A(p_i) = \{o_1, o_2, o_7, o_9, o_6\}$. However, it is difficult to obtain α , so EPT tries to maximize α . In order to further improve the efficiency of EPT, EPT* is equipped with a new **pivot selection algorithm (PSA)** [39].

pointer	$d(o_i, o_1)$	$d(o_i, o_6)$
o_1	0	6
o_2	$\sqrt{2}$	$\sqrt{26}$
o_4	$\sqrt{5}$	$\sqrt{29}$
o_3	$\sqrt{10}$	$\sqrt{10}$
o_5	2	4
o_6	6	0
o_7	$\sqrt{26}$	$\sqrt{2}$
o_8	$2\sqrt{5}$	$2\sqrt{2}$
o_9	5	$\sqrt{13}$

Fig. 16. CPT.

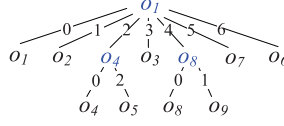


Fig. 17. BKT.

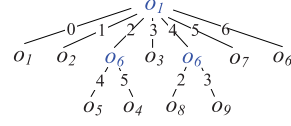


Fig. 18. FQT.

Discussion. The construction cost of EPT is $O(nlg)$, and its storage cost is $O(nl + ns + lgs)$, as EPT needs $O(ns)$ to store the data, $O(nl)$ to store the pre-computed distances, and $O(lgs)$ to store the pivots. EPT has l groups while each group contains g pivots, and thus, it needs $O(lgs)$ to store all the pivots. For each object, we will select the best pivot among each pivot group, and hence, it needs $O(nl)$ to store the pre-computed distances between objects and the best pivots. To get high quality pivots, the construction cost of EPT* is $O(nl_l c n_s)$, where l_c is the number of candidate pivots and n_s denotes the cardinality of the sample set. The storage cost of EPT* is $O(ns + nl + l_c s)$, as EPT* needs $O(ns)$ to store the data, $O(nl)$ to store the pre-computed distances, and $O(l_c s)$ to store all the pivots. EPT* achieves better similarity search performance than EPT contributed by the higher quality pivots selected by PSA. Nonetheless, the pivot selection is costly, and thus, the construction cost of EPT* is very high.

5.9 CPT

LAESA and EPT store the distance table and the data in main memory, and similarity query processing needs to scan the whole table. However, when the size of the dataset exceeds the capacity of the main memory, it is necessary to store the dataset on disk, and it becomes attractive to cluster the data to improve I/O efficiency. The CPT [94] uses an M-tree to cluster and store the objects on disk. The CPT consists of two parts, i.e., the distance table (depicted in Figure 16) and the M-tree (depicted in Figure 9). Note that, the distance table includes the pointers to the leaf entries in the M-tree, in order to enable loading of the corresponding objects for verification.

Discussion. The construction cost of CPT is $O(n(m..m^2) \log_m n + nl)$, as it needs $O(nl)$ to construct the table and $O(n(m..m^2) \log_m n)$ to build the M-tree. The storage cost of CPT is $O(ns + ns/m + nl + ls)$, as it needs $O(ns + ns/m)$ to store the M-tree, $O(nl)$ to store the pre-computed distances, and $O(ls)$ to store the pivots. Using CPT, we can avoid loading the whole dataset into main memory during query processing. However, CPT incurs the I/O cost to load objects from disk. In addition, the distance table is stored in main memory, meaning that the applicability of CPT is still limited to datasets for which the distance table fits in main memory.

5.10 BKT

BKT [25] is a tree structure designed for discrete distance functions. It chooses a pivot as the root, and maintains the objects having the distance i to the pivot in its i th sub-tree. Figure 17 gives an example BKT, constructed based on the objects from Figure 3 and the discrete distance function L_∞ -norm. However, for other metric index examples, Euclidean distance (L_2 -norm) is used. Note that, the continuous distance range can be partitioned into discrete ranges used for indexing. For example, if the continuous distance function range is $[0, 30]$, we can divide it into three disjoint ranges $[0, 10)$, $[10, 20)$, $[20, 30]$ in order to simulate the discrete distance function. Hence, BKT can be adapted to support both discrete and continuous distance functions.

Discussion. BKT is an unbalanced tree. The construction cost of BKT is $O(nl)$, where l denotes the height of BKT. To compare different pivot-based metric methods, we use the same number of pivots, and thus, the height of BKT is set to l in this article, and the construction cost is $O(nl)$ instead of $O(n \log n)$. However, we cannot use the same set of pivots for BKT. Like in the case of other pivot-based methods, BKT randomly selects the pivots for its sub-trees. If BKT uses the same pivots as other pivot-based metric indexes, it produces FQT as discussed below. The storage cost of BKT is $O(ns + \ln_d)$, where n_d denotes the number of discrete values in the domain of distance function $d()$. Here, we store the associated distances (e.g., 0, 1, 2, 3, 4, 5, 6 in Figure 17) for each sub-tree, which results in an additional storage cost of $O(\ln_d)$. To avoid empty sub-trees for large distance domains, each sub-tree covers a range of distance values, which are stored together.

5.11 FQ Family

Unlike BKT, FQT [14] utilizes the same pivot at the same level. Figure 18 shows an example of FQT, where o_1 is used for the first level, and o_6 is used for the second level. FQT is an unbalanced tree. Hence, FHQT [13] is proposed, where objects are stored in the leaves, and all the leaves are at the same level. In addition, FHQT can also be stored as FQA [35] using a table. Note that, FQA and LAESA are the same. Although FQT is designed for a discrete distance function, it can also be extended to support a continuous distance function, as in the case of BKT.

Discussion. Similar to BKT, the construction cost of FQT is $O(nl)$, and the storage cost of FQT is $O(ns + nl)$, where l is the height of FQT. This is because, in order to utilize the same set P of pivots as other pivot-based metric indexes, the height of FQT is set to the number of pivots, and $p_i \in P$ is set as the pivot for the i th level. With well-chosen pivots, FQT is expected to perform better than BKT.

5.12 VPT Family

VPT [138, 139, 149] is a binary tree. It chooses a pivot p as the root, and selects a medium value d_{med} so that the objects o with $d(o, p) \leq d_{med}$ are put in the left sub-tree, while the remaining objects are put in the right sub-tree. Figure 19(a) depicts an example of VPT, where L_∞ -norm is used. Note that, the pivots for the nodes at the same level can be different. In order to be able to compare the efficiency of different indexes using the same set of pivots, nodes of VPT at the same level share the same pivot. To support insertions and deletions of VPT, a dynamic version DVPT [64] is designed. VPT can be generalized to m -ary trees, yielding MVPT [20, 21]. Each time, MVPT selects $m-1$ medium values $d_1, d_2, \dots, d_{m-1}$. Figure 19(b) gives an example of MVPT, where m is 3. In addition, MVPT can also use multiple pivots (instead of using only one pivot) to partition each node.

Discussion. Unlike BKT and FQT, MVPT is a balanced tree. The construction costs of VPT and DVPT are $O(n \log_2 n)$, while the construction cost of MVPT is $O(n \log_m n)$. As m grows, the pruning ability first increases and then drops. This occurs because, with larger m values, more compact sub-trees are obtained at every tree level. Nevertheless, larger m values result in lower MVPT tree levels, indicating that fewer pivots are available for pruning. The storage costs of VPT, DVPT, and MVPT are all $O(ns)$. MVPT only stores medium values to partition the sub-trees, which incurs lower storage cost than BKT and FQT.

5.13 Omni-family

The Omni-family [22, 136] utilizes an existing secondary-memory index, e.g., a sequential file, a B^+ -tree, or an R-tree, to index the vectors after the pivot mapping. A sequential file stores the pre-computed distances of objects in the order of their identifiers; a B^+ -tree is used to index the pre-computed distances for each pivot; or an R-tree is used to index the pre-computed distances

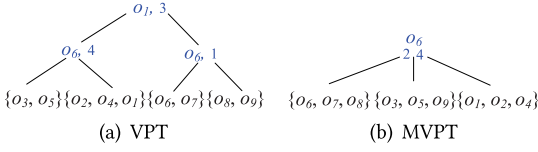


Fig. 19. Example of the VPT family.

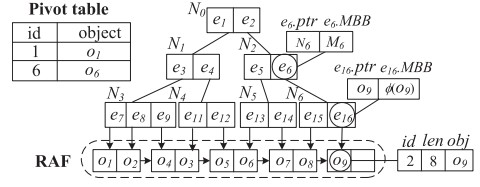


Fig. 20. Example of the Omni-tree.

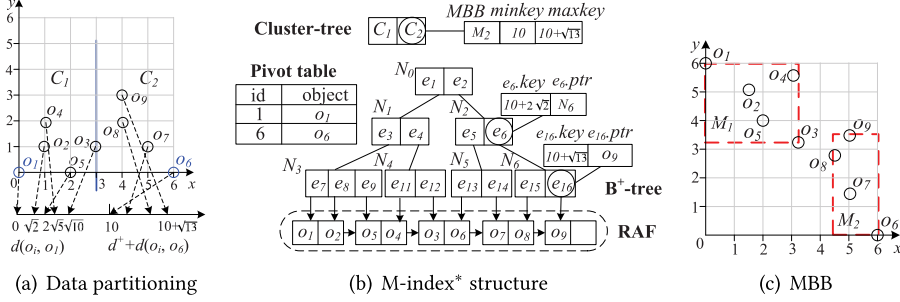


Fig. 21. Example of the M-index*.

w.r.t. all the pivots. Figure 20 depicts an example of the OmniR-tree, including the pivot table that stores the pivots, the R-tree that indexes the pre-computed distances, and the **random access file** (RAF) that stores the objects. Note that, unlike the M-tree family, the Omni-family uses a separate RAF to store the objects. This is done to avoid the impact of the object size.

Discussion. The Omni-family includes the Omni-sequential-file, the OmniB⁺-tree, and the OmniR-tree. The Omni-sequential-file can be regarded as LAESA stored on disk, which incurs substantial I/O during search as the data is unclustered. The OmniB⁺-tree needs one B⁺-tree for each pivot, resulting in redundant storage and I/O during search. The OmniR-tree utilizes MBBs to cluster the data, and uses pivot filtering to achieve high query efficiency. The construction cost of the OmniR-tree is $O(nml \log_m n)$, as it needs $O(nl)$ to do the pivot mapping and $O(nml \log_m n)$ to build the R-tree. The storage cost of the OmniR-tree is $O(ns + nl + nl/m + ls)$, as it needs $O(ns)$ to store the data in the RAF, $O(nl + nl/m)$ to store the pre-computed distances in the R-tree, and $O(ls)$ to store the pivots in the pivot table.

5.14 M-index

The M-index [107] combines generalized hyperplane partitioning and pivot mapping. Given a set P of pivots, each object o is assigned to the partition of its nearest pivot $p_i (i \in P)$, and is mapped to the real number $key(o) = d(p_i, o) + (i - 1) \times n_d$, where n_d is the maximum distance in a certain metric space. In Figure 21(a), if $P = \{o_1, o_6\}$, we obtain clusters C_1 and C_2 . After the partitioning, a cluster tree (as depicted in Figure 21(b)) is used to maintain the information of the clusters, a B⁺-tree is employed to index the mapped real numbers, and an RAF is used to store the objects with the pre-computed distances to all the pivots. If more pivots are used, the cluster-tree can be extended to a dynamic tree. To accelerate search using the M-index, the minimum and maximum mapped numbers of MBBs are added to the cluster tree in the M-index, yielding the M^{*}-index [39].

Discussion. The construction cost of the M-index is $\Omega(nl \log_l n/m) + O(mn \log_m n)$. Here, we need the optimal cost $\Omega(nl \log_l n/m)$ to construct the dynamic unbalanced cluster tree and cost $O(mn \log_m n)$ to construct the B⁺-tree, where m denotes the number of entries in each cluster or each B⁺-tree node. The storage cost of the M-index is $O(ns + nl + n + n/m + ls)$, as it needs $O(ns + nl)$

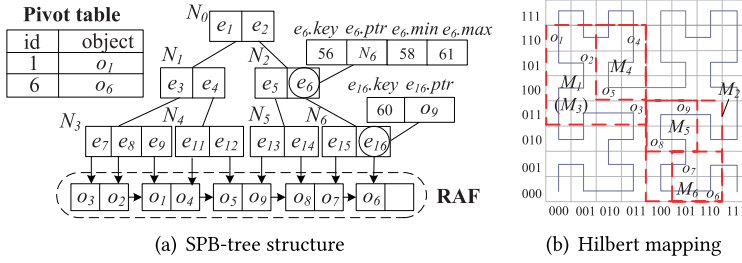


Fig. 22. Example of the SPB-tree.

cost to store the RAF, $O(n + n/m)$ cost to store the B^+ -tree, $O(n/m)$ cost to store the cluster tree, and $O(ls)$ cost to store the pivot table. When integrating the MBB into the cluster tree, the storage of the M-index* is increased to $O(ns + nl + n + n/m + nl/m + ls)$. However, the efficiency of MRQ and MkNNQ on the M-index* is improved. Since the M-index* can use both Lemmas 4.1 and 4.3 for pruning, it can achieve high performance in terms of distance computations. Nonetheless, it needs to visit B^+ -tree multiple times for all unpruned clusters, making the I/O cost relatively high.

5.15 SPB-tree

The SPB-tree [36, 37] utilizes the two-stage mapping, i.e., the pivot mapping and the **space-filling curve (SFC)** mapping, to map objects into SFC values (i.e., integers) while (to some extent) maintaining spatial proximity. Then, a B^+ -tree is used to store the SFC values. Figure 22 depicts an example of SPB-tree, where Figure 22(b) illustrates the Hilbert mapping after the pivot mapping. Each non-leaf entry e in the B^+ -tree stores the SFC values of the two corners of its MBB.

Discussion. The construction cost of the SPB-tree is $O(n(l^2 \cdot l^3) + n(m + l) \log_m n)$. In particular, the SPB-tree needs $O(nl^2)$ or $O(nl^3)$ to do the pivot mapping and the space filling curve mapping ($O(l)$ and $O(l^2)$ time are used for the Z-order curve and Hilbert curve, respectively, to transform each object), and takes $O(n(m + l) \log_m n)$ in building the B^+ -tree with MBB. The storage cost of the SPB-tree is $O(ns + n + n/m + ls)$, as it needs $O(ns)$ space to store the real data in RAF, $O(n + n/m)$ to store the mapped values in the B^+ -tree, and $O(ls)$ to store the pivot table. We employ the SFC mapping to reduce the storage cost and maintain spatial proximity, resulting in improved I/O and index storage costs. However, for continuous distance functions, the continuous distances are approximated by discrete ones to perform the SFC mapping, which decreases the pruning power. In addition, during similarity search, we need to do the Hilbert transformation to get the pre-computed distances; hence, additional CPU cost is incurred.

6 EXPERIMENTAL COMPARISON AMONG METRIC INDEXES

We detail the experimental setting and then cover experiments in main-memory and secondary-memory settings in turn. In addition, we conduct a significance evaluation of the metric indexes.

6.1 Experimental Settings

As discussed in Section 2.2, analytical studies that do not take the effect of pruning into account offer limited insight into the real-world search performance of metric indexes. Thus, we opt to compare experimentally 19 representative metric indexes, including BST (belonging to the BST family), GNAT, and EGNAT (belonging to the GHT family), SAT and DSACLT (belonging to the SAT family), M-tree and PM-tree (belonging to the M-tree family), LC (belonging to the LC family), MB^+ -tree and D-index (belonging to the D-index family), LAESA (belonging to the AESA family), EPT*, CPT, BKT, FQT (belonging to the FQ family), MVPT (belonging to the VPT family),

Table 5. Datasets Used in the Experiments

Datasets	Cardinality	Dimensionality	Intrinsic Dimensionality	Measurement
<i>LA</i>	1,073,727	2	5.4	L_2 -norm
<i>Words</i>	611,756	1~34	1.2	Edit distance
<i>Color</i>	1,000,000	282	6.5	L_1 -norm
<i>Synthetic</i>	1,000,000	20	6.6	L_∞ -norm

Table 6. Parameter Setting

Parameter	Values	Default
Number of pivots l	3, 5, 10, 15, 20	5
Search radius r (selectivity of range queries)	2%, 4%, 8%, 16%, 32%	8%
Number of k	5, 10, 20, 50, 100	20

OmniR-tree (belonging to the Omni-family), SPB-tree, and M-index*. We implemented all indexes and associated similarity search algorithms in C++, and experiments were conducted on an Intel Core i7-7700 3.6 GHz PC with 16 GB memory. All code is publicly available.⁷

We employ three real datasets, namely, *LA*, *Words*, and *Color*. *LA*⁸ consists of geographical locations in Los Angeles, and the L_2 -norm is utilized to measure similarity. *Words*⁹ contains proper nouns, acronyms, and compound words taken from the Moby project, and the edit distance is used to compute the distances between words. *Color*¹⁰ consists of standard MPEG-7 image features extracted from Flickr, and the similarity between two features is measured by the L_1 -norm. In addition, we generate a dataset *Synthetic*, where the values are integers and the L_∞ -norm is employed. More specifically, in *Synthetic*, 5 of 20 dimension values are generated randomly, while the remaining values are linear combinations of the previous ones. Table 5 summarizes statistics of the datasets.

We investigate the similarity query performance using the indexes when varying the parameters listed in Table 6.

- The number l of pivots is an integer in $[1, +\infty)$, and we find that a small l yields the best performance for all four datasets. Hence, we chose five representative values (specifically 3, 5, 10, 15, 20) for l .
- Following a previous study [136], we use the selectivity of metric range queries to set the search radius r that controls the search region. In particular, the value of radius r denotes the percentage of objects in the dataset that are result objects of a metric range query. As different datasets have different distance ranges, it is difficult to use the same value across different datasets. Thus, we use selectivity values (i.e., 2%, 4%, 8%, 16%, and 32%) instead of fixed values (e.g., 1 km and 2 km) to better evaluate the performance of metric range queries.
- The value k in the $MkNN$ query is an integer in $[1, +\infty)$. In real-life applications, a small number of result objects is usually retrieved by an $MkNN$ query. For example, in recommendation systems, we cannot provide too many recommendations for users. Thus, we vary k from 5 to 100, which aligns with previous studies [35, 136].

In each experiment, one parameter is varied, and the others are fixed at their default values. As explained in Section 2.2, the main performance metrics include the number of PA , the number of distance computations (*compdists*), and the running time. Each reported measurement is an

⁷Code is available at https://github.com/ZJU-DAILY/Metric_Index.

⁸*LA* is available at [http://www.dbs.informatik.uni-muenchen.de/\\$\sim\\$seidl](http://www.dbs.informatik.uni-muenchen.de/$\sim$seidl).

⁹*Words* is available at <http://icon.shef.ac.uk/Moby/>.

¹⁰*Color* is available at <http://cophir.isti.cnr.it/>.

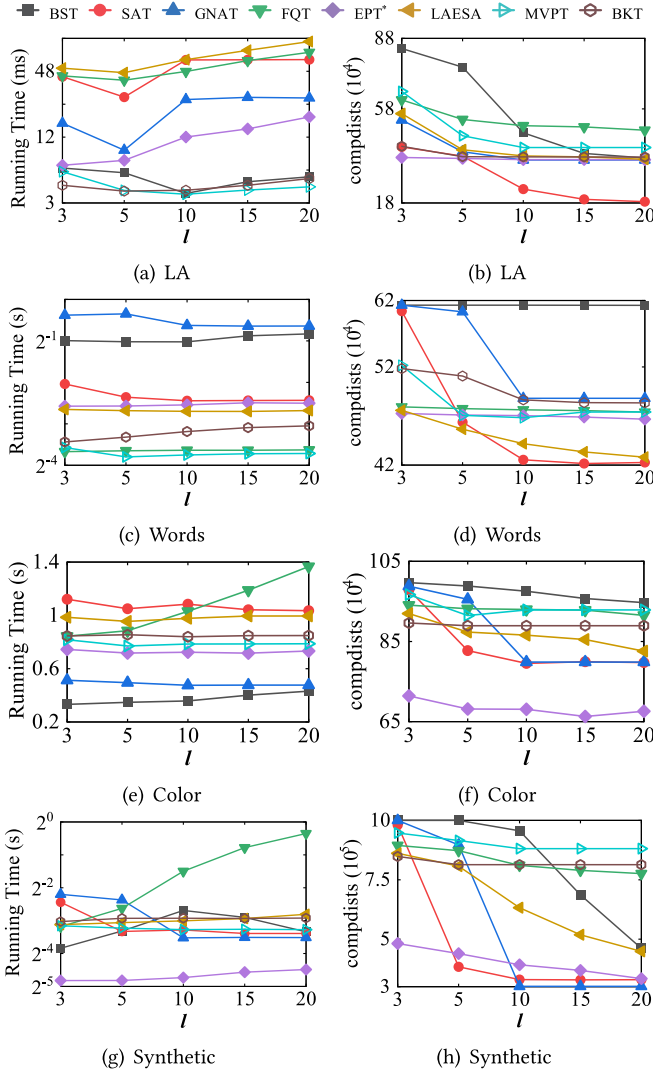


Fig. 23. Main-memory based metric index comparison using MRQ queries.

average over 100 random queries. To facilitate a fair comparison, we use the same set of random queries for all indexes.

When comparing the performance of pivot-based methods, their use of different pivot selection strategies renders the comparison challenging. For example, the OmniR-tree [136] utilizes the HF algorithm to select outliers as pivots, while the SPB-tree uses the HFI algorithm to select pivots that maximize the similarity between the original metric space and the vector space obtained by using the pivots. Since the similarity query performance depends highly on the pivots used [8, 26], we utilize the same pivot selection strategy for the pivot-based indexes. Specifically, we use the HFI algorithm [36, 39]. This does, however, not apply to the EPT*, BKT, and GNAT families. As discussed in Sections 5.1, 5.8, and 5.10, GNAT uses centers in the same level as pivots, EPT* utilizes different pivots for different objects, and BKT needs to select pivots randomly in its sub-trees.

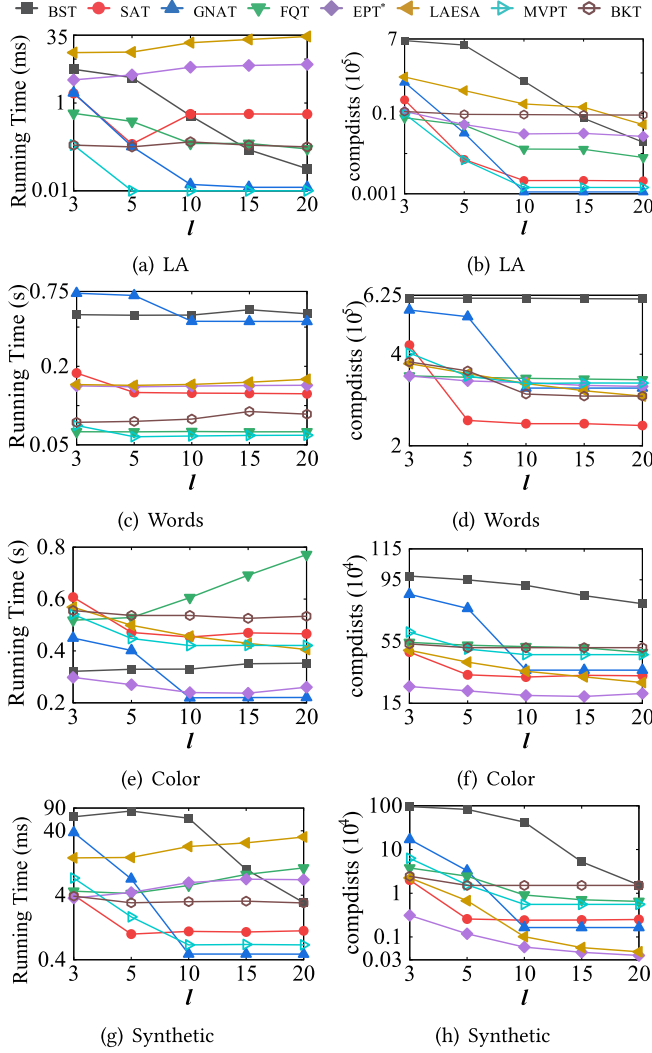


Fig. 24. Main-memory based metric index comparison using $MkNN$ queries.

6.2 Comparison among Main-memory Based Metric Indexes

We compare the performance of the main-memory based metric indexes using MRQ and $MkNN$ queries, which includes compact-partitioning based indexes (i.e., BST and SAT), pivot-based indexes (i.e., LAESA, EPT*, BKT, FQT, and MVPT), and the hybrid index GNAT. When comparing the pivot-based indexes and compact-partitioning based indexes, the index pruning ability depends on the number of pivots and the number of centers, respectively. Hence, we set the number of pivots used in the pivot-based indexes equaling to the height of compact-partitioning based methods. Then, the number of pivots and the number of centers used for pruning each object in pivot-based and compact-partitioning methods, respectively, are the same. GNAT and MVPT are multi-arity trees. Here, we set arity to 5, which yields the best results across a range of values tested.

Figures 23 and 24 show the MRQ and $MkNN$ performance (*compdicts* and running time) on four datasets, respectively. The first observation is that, the query cost, *compdicts* as well as running

Table 7. Main-memory-based Metric Indexes Ranking

Main-memory based Metric Index	Compdist ranking		Running time ranking	
	MRQ	MkNN	MRQ	MkNN
BST	8	8	3	7
SAT	1	1	6	4
LAESA	4	4	7	8
EPT*	2	2	2	4
BKT	5	6	4	3
FQT	6	7	7	6
MVPT	6	5	1	1
GNAT	3	2	4	2

time, first drops and then remains stable or increases when the number of pivots or the tree height increase. On the one hand, the more pivots/the higher the tree, the stronger pruning ability. On the other hand, using more pivots increases the CPU cost of filtering the search space. Hence, the best number of pivots for an index depends on the distance distribution of the dataset and the structure of the index. Table 7 summarizes the performance metric (including *compdist*s and running time) ranking of all main-memory based indexes on the four datasets using MRQ and MkNN queries.

Compdist Performance Analysis. As summarized in Table 7, SAT, EPT*, and GNAT perform best in terms of distance computations, while BST and FQT perform the worst. The number of distance computations depends on the selected pivots and the pivot based filtering and validation techniques used. However, to achieve a fair comparison among the index structures, we use the same set pivots for all the metric indexes except EPT* and BST as explained earlier. Thus, the reasons behind the distance computation performance observation are that: (1) EPT* selects different pivots for each object in order to improve its pruning, while the other indexes use the same set of pivots for all objects in the dataset; (2) GNAT is a hybrid index that uses two different pivot-based filtering techniques, which improves its pruning ability; (3) SAT utilizes its ancestors for pruning while the others use only the centers at the same level for pruning, which improves the pruning ability of SAT; (4) BST uses random pivots, which yields relatively weak filtering; and (5) FQT is a unbalanced tree, indicating that relatively few pivots are used for pruning. As stated in previous work, the pivot-based metric indexes can achieve better performance in terms of distance computations when compared with compact-partitioning based methods [35]. However, if we use the same set of pivots and the same pruning and validation techniques, the number of distance computations across these two categories are similar. A key reason why the distance computation performance of the pivot-based metric indexes better is that they (e.g., LAESA, EPT*) can support unlimited number of pivots, while the pivot number for compact-partitioning methods is limited by the dataset size (i.e., if little data is left, no further partitions/no additional centers are needed to build the index).

CPU Performance Analysis. As summarized in Table 7, MVPT, GNAT, and EPT* achieve the best performance in terms of running time, while LAESA and FQT perform the worst. Although LAESA needs few distance computations, the CPU performance is bad. This is because it scans the entire table without opportunities for bulk pruning. Next, MVPT and GNAT achieve better CPU performance as they are balanced trees while the others (SAT, BKT, and FQT) are unbalanced. Although BST employs a balanced tree structure, its CPU cost is high because its tree is a binary tree and is weak in terms of pruning. Hence, the CPU performance not only depends on the number of distance computations needed during the search, but also on the index structure.

Table 8. Secondary-memory Based Metric Indexes Ranking

Secondary-memory based Metric Index	PA ranking		Compdists ranking		Running time ranking	
	MRQ	MkNN	MRQ	MkNN	MRQ	MkNN
LC	1	1	1	5	1	1
DSACLT	7	9	5	8	2	8
M-Tree	9	5	9	9	4	2
MB ⁺ -tree	4	7	10	11	10	11
OmniR-tree	5	4	8	2	7	3
SPB-tree	2	2	6	4	5	7
D-index	3	3	11	10	2	6
EGNAT	11	11	2	7	11	10
PM-tree	9	5	2	3	5	4
CPT	8	7	7	6	9	5
M-index*	6	10	4	1	8	9

6.3 Comparison among Disk-based Metric Indexes

We proceed to compare the MRQ and MkNN query performance of all secondary-memory based metric indexes. These indexes include compact-partitioning based methods (i.e., DSACLT, LC, MB⁺-tree, and M-tree), pivot-based methods (i.e., OmniR-tree and SPB-tree), and hybrid methods (i.e., D-index, EGNAT, PM-tree, CPT, and M-index*). For the secondary-memory based indexes (the B⁺-tree, R-tree, and M-tree, etc.), the tree height depends on the page/node size. We fix the page size to 4 KB. The index height can then be calculated based on the index structure and the data distribution. However, CPT, the M-tree, the PM-tree, LC, DSACLT, and EGNAT all store the data objects directly in their index structures. Thus, they need a large page size for high-dimensional data, and they are configured to use 40 KB pages when applied to *Color* and *Synthetic* datasets as default. However, a 40 KB page can be regarded as 10 4 KB pages for one tree node or cluster. In addition, the number of pivots is fixed to 5, and all the pivot-based methods and hybrid methods (except EGNAT) use the same set of pivots for fair comparison. This is because, EGNAT uses the centers in each tree-level as the pivots for this tree-level due to its design.

Figures 25 and 26 show the MRQ and MkNN query performance of the secondary-memory based metric indexes when applied to the four datasets, respectively. As expected, the query cost (including *PA*, *compdists*, and running time) increases with the growth of k and r due to the resulting larger search spaces. Table 8 displays the performance metric (including *PA*, *compdists*, and running time) ranking of the secondary-memory based metric indexes over four datasets using MRQ and MkNN queries. In the following, we provide observations on the ranking with respect to three different performance metrics.

I/O Performance Analysis. As summarized in Table 8, LC, the SPB-tree, and the D-index achieve the best performance in terms of *PA*, followed by the OmniR-tree, the MB⁺-tree, the M-tree, the PM-tree, and CPT, while EGNAT, DSACLT, and the M-index* perform the worst. The I/O cost depends on whether the data is well clustered and on *compdists* needed during search. The compact-partitioning based methods can achieve better I/O performance compared with the pivot-based methods and the hybrid methods, because they are able to cluster the data well and because the pivot-based and hybrid methods need to store additionally large number of pre-computed distances. As LC is partitioning-based method that can cluster the data well, they achieve good I/O performance. Although the M-tree, the MB⁺-tree, and DSACLT cluster the data in compact partitions, their *compdists* during search are high (i.e., corresponding to low pruning ability). The

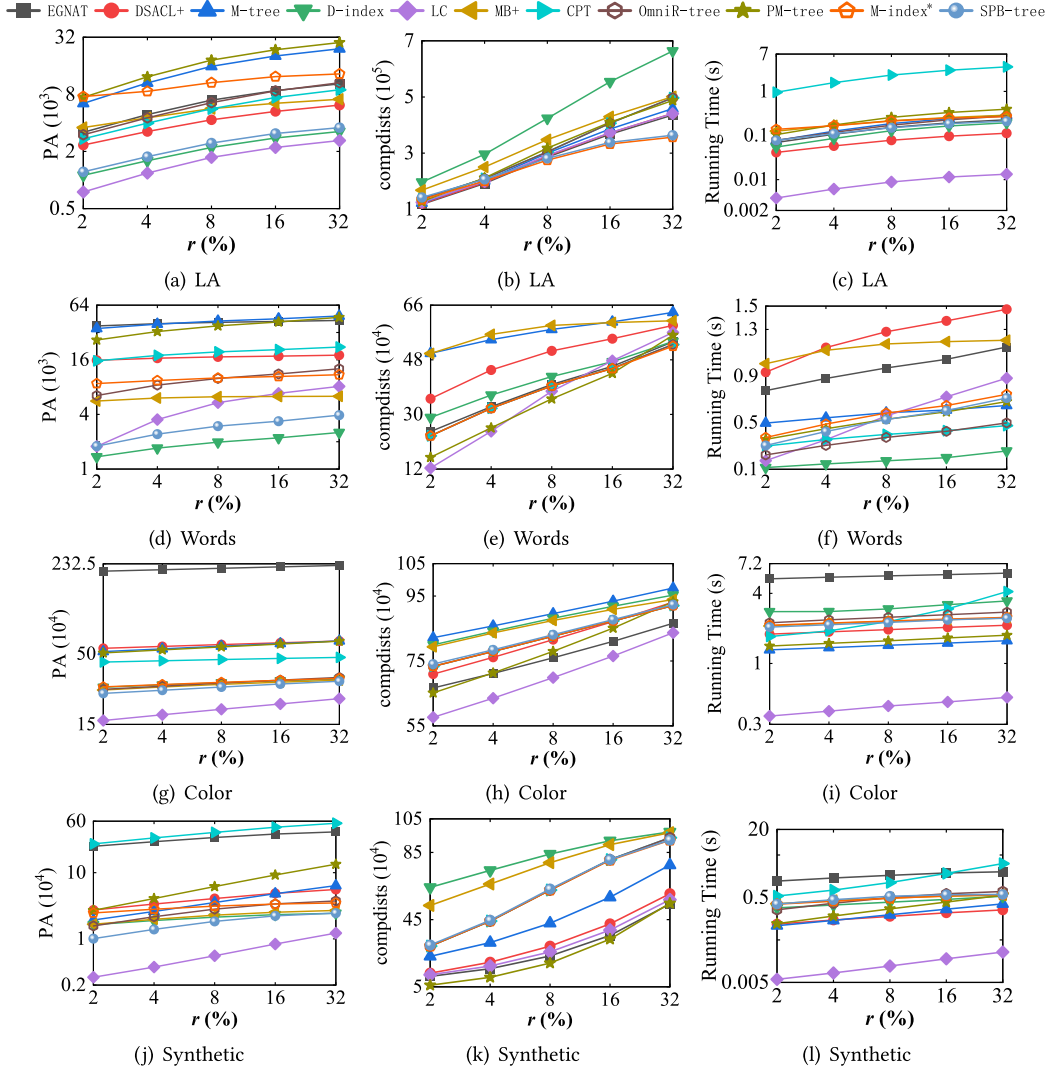


Fig. 25. Secondary-memory based metric index comparison using MRQ queries.

SPB-tree, a pivot-based method, uses a space-filling curve to cluster the data and reduce the storage needed for pre-computed distances. Thus, it achieves good I/O performance.

Compids Performance Analysis. As summarized in Table 8, the M-index*, the OmniR-tree, the PM-tree, and the SPB-tree achieve the best performance in terms of *compids* for MkNN queries, while LC, EGNAT, the PM-tree, and the M-index* perform the best for MRQ queries. Thus, the secondary-memory based index performance varies slightly across the two types of queries. This is because the search radius for MkNN queries is very small, while the search radius for MRQ is large in the experiments. More specifically, LC is constructed and searched in order of data occurrences, while the other indexes can search in best-first order to find NNs quickly and reduce the search space quickly. Hence, LC is better for MRQ with large search radius when compared to MkNN queries with small search radius. Overall, the pivot-based and hybrid methods achieve better *compids* performance than the compact-partitioning methods. This is because we employ

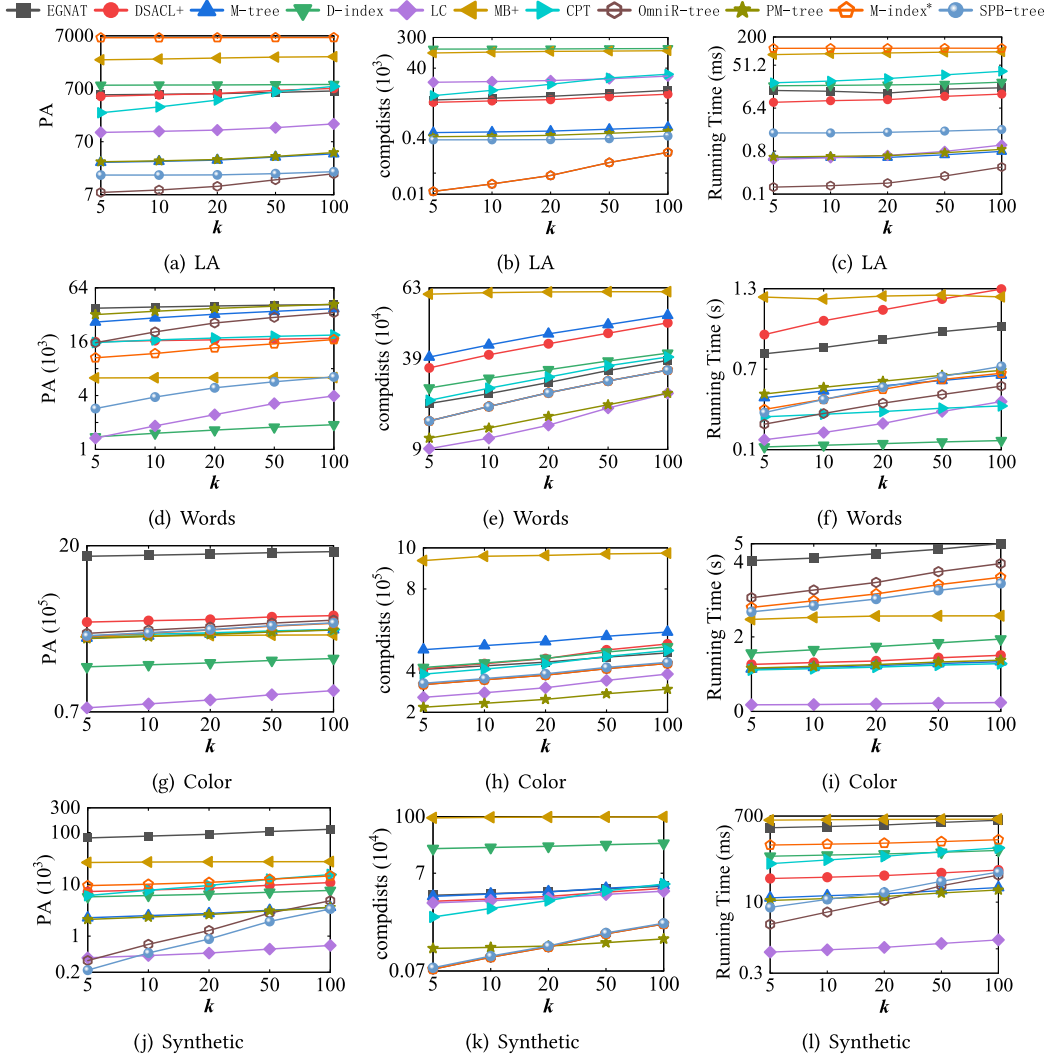


Fig. 26. Secondary-memory based metric index comparison using MkNN queries.

high-quality pivots in pivot-based and hybrid methods, while the pruning ability of centers in compact-partitioning methods is weak. Also note that the construction cost of LC, a compact-partitioning method, is $O(n^2)$, enabling it to cluster the data well to trade higher construction cost for better query performance. However, the construction cost of LC is excessive when the cardinality of the dataset is very large. As a result, LC is not a practical choice. Next, although the D-index is a hybrid method that uses pre-computed distances to prune the search space, the hashing partitioning makes it hard to control the quality of clusters, which depends on the split parameter ρ and the search radius.

Running Time Performance Analysis. The running time depends on PA, compdist, and the CPU cost of pruning, which is used to evaluate the overall performance. As summarized in Table 8, LC and the M-tree perform the best in terms of running time, followed by the OmniR-tree, the PM-tree, the D-index, the SPB-tree, and DSACL+, while EGNAT, the MB+-tree, the M-index*, and CPT

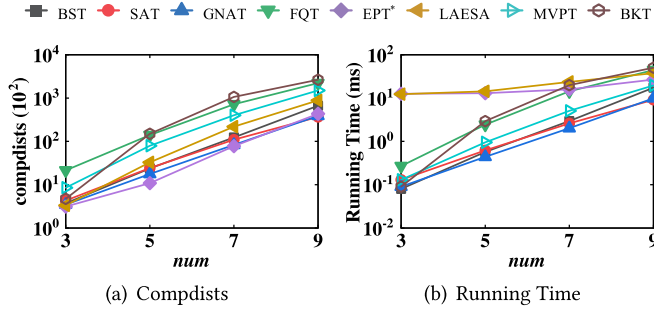


Fig. 27. MkNNQ performance using main-memory based metric indexes vs. *num* on synthetic datasets.

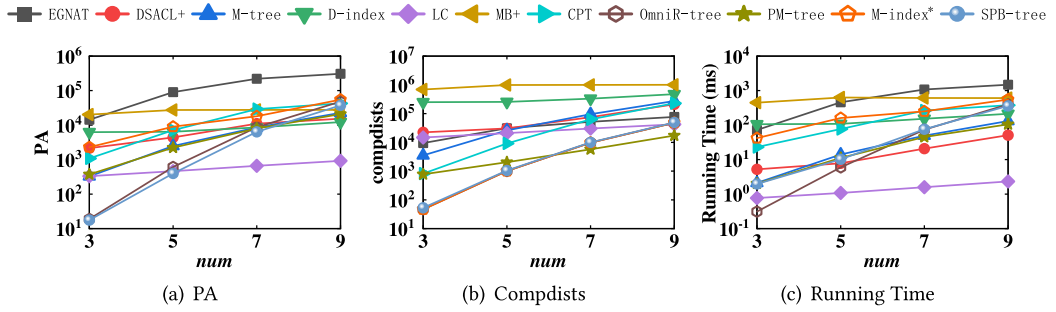


Fig. 28. MkNNQ performance using secondary-memory based metric indexes vs. *num* on synthetic datasets.

perform the worst. Here, LC has good running time performance, because it performs well in terms of both *PA* and *compdists*. The M-index* and EGNAT, which have good *compdists* performance, exhibit weak running time performance due to high I/O cost during search. Note that, although the PM-tree achieve better *compdists* performance and similar *PA* performance compared with the M-tree, the M-tree performs slightly better in terms of running time performance due to the relatively high pruning CPU cost required by the PM-tree. Although, the SPB-tree shows a very good performance on *PA*, it incurs additional CPU cost (i.e., space filling curve transformation) during the search, resulting in a relatively high CPU cost.

6.4 Effect of Intrinsic Dimensionality

In order to vary the ID, we change the dimensionality of *Synthetic* dataset (denoted as *num*) among 3, 5, 7, and 9. Specifically, *num* of the 20-dimension values in *Synthetic* is generated randomly, while the remaining values are linear combinations of the previous ones. The corresponding IDs are 3.7, 6.8, 10.5, and 14.1, respectively.

Figure 27 plots the performance results of MkNN queries using main-memory metric indexes by varying *num*, while all other parameters are set to their default values. As can be observed, the running times of LAESA and EPT* are less sensitive to *num*. This is because, LAESA and EPT* are stored as tables, while the others are stored as trees. We need to scan two entire indexes (LAESA and EPT*) during search for any value of *num*. However, the performance of the tree structures (BST, SAT, BKT, FQT, MVPT, and GNAT) degrades as *num* increases (i.e., more sub-trees are visited).

Figure 28 illustrates the performance results of MkNN queries using secondary-memory metric indexes. The first observation is that pivot-based indexes (i.e., the OmniR-tree, the SPB-tree, and

the M-index*) are more sensitive to *num* when compared with the compact-partitioning methods. The reason is that, the number of pivots is fixed in our experiments, which limits the pruning capabilities when the ID increases. More pivots are required with the growth of the ID. The second observation is that LC and the MB⁺-tree are most stable across all performance metrics as *num* grows. This is because, (i) LC is a LC, where the query is performed by sequential scanning of clusters; and (ii) the query using the MB⁺-tree needs to verify almost all data objects. The third observation is that the OmniR-tree and the SPB-tree are the indexes that are most sensitive to *num*. The reason is that, the performance of the R-tree (used by the OmniR-tree) and the spatial locality of spacing-filling curve (used by the SPB-tree) degrades with the growth of the ID.

7 SUMMARY AND RESEARCH DIRECTIONS

We have surveyed metric indexes that aim to accelerate exact similarity search, which can be classified into three categories, i.e., pivot-based methods, compact-partitioning based methods, and hybrid methods. Pivot-based methods store pre-computed distances between well selected pivots and data objects, utilizing one of three types of data structures (table, tree, and secondary-memory based index). Thus, pivot-based indexes can be further divided into three types according to the data structure used to store pre-computed distances. Then, compact-partitioning based methods use one of four different types of partitioning techniques (ball partitioning, hyperplane partitioning, hash partitioning, and hybrid partitioning) to cluster the data. Thus, compact-partitioning based methods can be further divided into four types according to the partitioning technique used.

Next, we have covered all pruning and validation techniques based on pivots or centers that are employed to accelerate similarity search using metric indexes. In addition, we have covered time and space complexities for metric index construction. Finally, we have reported on experimental analyses of similarity search using the metric indexes. The resulting findings, as below, enable users to select the indexes that best support the intended use cases:

- (1) For small datasets with little need for scalability, the main-memory based metric indexes can be chosen.
 - a. For complex distance functions (i.e., the distance computation cost is the dominance CPU cost), EPT*, SAT, and GNAT are the best choices, among which SAT has the lowest construction cost, followed by GNAT and EPT*.
 - b. For simple distance functions, MVPT, GNAT, and EPT* are the best choices, where the MVPT has lower construction cost than that of GNAT and EPT*.
- (2) For large datasets that call for scalability, the disk-based metric indexes can be chosen.
 - a. For complex distance functions, the PM-tree, the SPB-tree and the OmniR-tree are the best choices. They achieve good performance in terms of *compdists* and have relatively good I/O performance. We not recommend LC due to its huge construction cost for large datasets.
 - b. For simple distance functions, the M-tree, the PM-tree, and D-index are the best choices. They achieve relatively good performance in terms of *PA* and incur little additional CPU cost for pruning.

Although many metric indexes have been proposed, a number of open issues require further attention. Possible future directions of metric indexes are summarized below:

- (1) Search space pruning is based on pivots when using pivot-based methods while it is based on centers for compact-partitioning based methods. To improve pruning performance, it is of interest to study how to select high-quality pivots or centers, especially for compact-partitioning based methods.

- (2) To further improve the performance of metric indexes, intelligent metric indexing can be considered that exploits machine learning techniques. In addition, distributed platforms can be leveraged, and new approximation optimizations can be pursued.
- (3) As metric spaces only utilize the triangle inequality to accelerate search, possible directions for further work are how to improve the search efficiency by integrating specific characteristics of metric data and how to achieve high search efficiency if the triangle inequality is not fully satisfied by desirable distance functions.
- (4) As expectations of privacy increase, privacy concerns exist wherever personally identifiable information is collected, stored, and used. Hence, the last but not the least direction for future work is to offer privacy protection in metric indexing.

REFERENCES

- [1] Charu C. Aggarwal and Philip S. Yu. 2001. Outlier detection for high dimensional data. In *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*. 37–46.
- [2] Jurandy Almeida, Ricardo da S. Torres, and Neucimar J. Leite. 2010. BP-tree: An efficient index for similarity search in high-dimensional metric spaces. In *Proceedings of the 19th ACM International Conference on Information and Knowledge Management*. 1365–1368.
- [3] Jurandy Almeida, Eduardo Valle, Ricardo da S. Torres, and Neucimar J. Leite. 2010. DAHC-tree: An effective index for approximate search in high-dimensional metric spaces. *Journal of Information and Data Management* 1, 3 (2010), 375–390.
- [4] Giuseppe Amato, Claudio Gennaro, and Pasquale Savino. 2014. MI-file: Using inverted files for scalable approximate similarity search. *Multimedia Tools and Applications* 71, 3 (2014), 1333–1362.
- [5] Laurent Amsaleg, Oussama Chelly, Michael E. Houle, Ken-Ichi Kawarabayashi, Miloš Radovanović, and Weeris Treeratanajaru. 2019. Intrinsic dimensionality estimation within tight localities. In *Proceedings of the 2019 SIAM International Conference on Data Mining*. 181–189.
- [6] Fabrizio Angiulli and Fabio Fasseti. 2012. Indexing uncertain data in general metric spaces. *IEEE Transactions on Knowledge and Data Engineering* 24, 9 (2012), 1640–1657.
- [7] Arvind Arasu, Venkatesh Ganti, and Raghav Kaushik. 2006. Efficient exact set-similarity joins. In *Proceedings of the 32nd International Conference on Very Large Data Bases*. 918–929.
- [8] Luis G. Ares, Nieves R. Brisaboa, María F. Esteller, Oscar Pedreira, and Angeles S. Places. 2009. Optimal pivots to minimize the index size for metric access methods. In *Proceedings of the 2009 2nd International Workshop on Similarity Search and Applications*. 74–80.
- [9] Luis G. Ares, Nieves R. Brisaboa, Alberto Ordóñez Pereira, and Oscar Pedreira. 2012. Efficient similarity search in metric spaces with cluster reduction. In *Proceedings of the International Conference on Similarity Search and Applications*. 70–84.
- [10] Lior Aronovich and Israel Spiegler. 2007. CM-tree: A dynamic clustered index for similarity search in metric databases. *Data & Knowledge Engineering* 63, 3 (2007), 919–946.
- [11] Vassilis Athitsos, Jonathan Alon, Stan Sclaroff, and George Kollios. 2007. Boostmap: An embedding method for efficient nearest neighbor retrieval. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 30, 1 (2007), 89–104.
- [12] Vassilis Athitsos, Michalis Potamias, Panagiotis Papapetrou, and George Kollios. 2008. Nearest neighbor retrieval using distance-based hashing. In *Proceedings of the 2008 IEEE 24th International Conference on Data Engineering*. 327–336.
- [13] Ricardo Baeza-Yates. 1997. Searching: An algorithmic tour. *Encyclopedia of Computer Science and Technology* 37 (1997), 331–359.
- [14] Ricardo Baeza-Yates, Walter Cunto, Udi Manber, and Sun Wu. 1994. Proximity matching using fixed-queries trees. In *Proceedings of the Annual Symposium on Combinatorial Pattern Matching*. 198–212.
- [15] Marcelo Barroso, Nora Reyes, and Rodrigo Paredes. 2010. Enlarging nodes to improve dynamic spatial approximation trees. In *Proceedings of the 3rd International Conference on Similarity Search and Applications*. 41–48.
- [16] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-tree: An efficient and robust access method for points and rectangles. In *Proceedings of the SIGMOD Record*, Vol. 19. 322–331.
- [17] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Communication of the ACM* 18, 9 (1975), 509–517.
- [18] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. 1999. When is “nearest neighbor” meaningful?. In *Proceedings of the International Conference on Database Theory*. 217–235.

- [19] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *Proceedings of the 23rd International Conference on Machine Learning*. 97–104.
- [20] Tolga Bozkaya and Meral Ozsoyoglu. 1997. Distance-based indexing for high-dimensional metric spaces. In *Proceedings of the 1997 ACM SIGMOD International Conference on Management of Data*. 357–368.
- [21] Tolga Bozkaya and Meral Ozsoyoglu. 1999. Indexing large metric spaces for similarity search queries. *ACM Transactions on Database Systems* 24, 3 (1999), 361–404.
- [22] Svein Erik Bratsberg and Magnus Lie Hetland. 2012. Dynamic optimization of queries in pivot-based indexing. *Multimedia Tools and Applications* 60, 2 (2012), 261–275.
- [23] Sergey Brin. 1995. Near neighbor search in large metric spaces. In *Proceedings of the 21th International Conference on Very Large Data Bases* (1995), 574–584.
- [24] Luis Britos, A. Marcela Printista, and Nora Reyes. 2012. DSACL⁺-tree: A dynamic data structure for similarity search in secondary memory. In *Proceedings of the International Conference on Similarity Search and Applications*. 116–131.
- [25] Walter A. Burkhard and Robert M. Keller. 1973. Some approaches to best-match file searching. *Communication of the ACM* 16, 4 (1973), 230–236.
- [26] Benjamin Bustos, Gonzalo Navarro, and Edgar Chávez. 2003. Pivot selection techniques for proximity searching in metric spaces. *Pattern Recognition Letters* 24, 14 (2003), 2357–2366.
- [27] Benjamin Bustos and Tomáš Skopal. 2006. Dynamic similarity search in multi-metric spaces. In *Proceedings of the 8th ACM International Workshop on Multimedia Information Retrieval*. 137–146.
- [28] Domenico Cantone, Alfredo Ferro, Alfredo Pulvirenti, Diego Reforgiato Recupero, and Dennis Shasha. 2005. Antipole tree indexing to support range search and k-nearest neighbor search in metric spaces. *IEEE Transactions on Knowledge and Data Engineering* 17, 4 (2005), 535–550.
- [29] Caio César Mori Carêlo, Ives Rene Venturini Pola, Ricardo Rodrigues Ciferri, Agma Juci Machado Traina, Caetano Traina Jr, and Cristina Dutra de Aguiar Ciferri. 2011. Slicing the metric space to provide quick indexing of complex data in the main memory. *Information Systems* 36, 1 (2011), 79–98.
- [30] Caio CÚsar Mori CarÚlo, Ives RenÚ Venturini Pola, Ricardo Rodrigues Ciferri, Agma Juci Machado Traina, Cristina Dutra de Aguiar Ciferri, et al. 2009. The onion-tree: Quick indexing of complex data in the main memory. In *Proceedings of the East European Conference on Advances in Databases and Information Systems*. 235–252.
- [31] Edgar Chávez, Karina Figueroa, and Gonzalo Navarro. 2008. Effective proximity retrieval by ordering permutations. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 30, 9 (2008), 1647–1658.
- [32] Edgar Chávez, Verónica Luduena, Nora Reyes, and Patricia Roggero. 2016. Faster proximity searching with the distal SAT. *Information Systems* 59, July (2016), 15–47.
- [33] Edgar Chávez and Gonzalo Navarro. 2000. An effective clustering algorithm to index high dimensional metric spaces. In *Proceedings of the 7th International Symposium on String Processing and Information Retrieval*. 75–86.
- [34] Edgar Chávez and Gonzalo Navarro. 2005. A compact space decomposition for effective metric indexing. *Pattern Recognition Letters* 26, 9 (2005), 1363–1376.
- [35] Edgar Chávez, Gonzalo Navarro, Ricardo Baeza-Yates, and José Luis Marroquín. 2001. Searching in metric spaces. *ACM Computing Survey* 33, 3 (2001), 273–321.
- [36] Lu Chen, Yunjun Gao, Xinhao Li, Christian S. Jensen, and Gang Chen. 2015. Efficient metric indexing for similarity search. In *Proceedings of the 2015 IEEE 31st International Conference on Data Engineering*. 591–602.
- [37] Lu Chen, Yunjun Gao, Xinhao Li, Christian S. Jensen, and Gang Chen. 2015. Efficient metric indexing for similarity search and similarity joins. *IEEE Transactions on Knowledge and Data Engineering* 29, 3 (2015), 556–571.
- [38] Lu Chen, Yunjun Gao, Xinhao Li, Christian S. Jensen, Gang Chen, and Baihua Zheng. 2015. Indexing metric uncertain data for range queries. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 951–965.
- [39] Lu Chen, Yunjun Gao, Baihua Zheng, Christian S. Jensen, Hanyu Yang, and Keyu Yang. 2017. Pivot-based metric indexing. *PVLDB* 10, 10 (2017), 1058–1069.
- [40] Lu Chen, Yunjun Gao, Aoxiao Zhong, Christian S. Jensen, Gang Chen, and Baihua Zheng. 2017. Indexing metric uncertain data for range queries and range joins. *The VLDB Journal* 26, 4 (2017), 585–610.
- [41] Kenneth Ward Church. 2017. Word2Vec. *Natural Language Engineering* 23, 1 (2017), 155–162.
- [42] Paolo Ciaccia and Marco Patella. 1998. Bulk loading the M-tree. In *Proceedings of the 9th Australasian Database Conference*. 15–26.
- [43] Paolo Ciaccia and Marco Patella. 2000. The M²-tree: Processing complex multi-feature queries with just one index. In *Proceedings of the DELOS Workshop*.
- [44] Paolo Ciaccia and Marco Patella. 2002. Searching in metric spaces with user-defined and approximate distances. *ACM Transactions on Database Systems* 27, 4 (2002), 398–437.
- [45] Paolo Ciaccia and Marco Patella. 2017. The power of distance distributions: Cost models and scheduling policies for quality controlled similarity queries. In *Proceedings of the International Conference on Similarity Search and Applications*. 3–16.

- [46] Paolo Ciaccia, Marco Patella, and Pavel Zezula. 1997. M-tree: An efficient access method for similarity search in metric spaces. In *Proceedings of the VLDB*. 426–435.
- [47] Kenneth L. Clarkson. 2006. Nearest-neighbor searching and metric space dimensions. In *Nearest-neighbor Methods for Learning and Vision: Theory and Practice* (2006), 15–59.
- [48] Richard Connor. 2016. Reference point hyperplane trees. In *Proceedings of the International Conference on Similarity Search and Applications*. 65–78.
- [49] Richard Connor, Franco Alberto Cardillo, Lucia Vadicamo, and Fausto Rabitti. 2016. Hilbert exclusion: Improved metric search through finite isometric embeddings. *ACM Transactions on Information Systems* 35, 3 (2016), 1–27.
- [50] Richard Connor and Alan Dearn. 2018. Querying metric spaces with bit operations. In *Proceedings of the International Conference on Similarity Search and Applications*. 33–46.
- [51] Richard Connor, Lucia Vadicamo, Franco Alberto Cardillo, and Fausto Rabitti. 2019. Supermetric search. *Information Systems* 80, February (2019), 108–123.
- [52] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: Simplified data processing on large clusters. *Communication of the ACM* 51, 1 (2008), 107–113.
- [53] FKHA Dehne and Hartmut Noltemeier. 1987. Voronoi trees and clustering problems. *Information Systems* 12, 2 (1987), 171–175.
- [54] Frank Dehne and Hartmut Noltemeier. 1988. Voronoi trees and clustering problems. In *Proceedings of the Syntactic and Structural Pattern Recognition*. 185–194.
- [55] Vlastislav Dohnal. 2004. An access structure for similarity search in metric spaces. In *Proceedings of the International Conference on Extending Database Technology*. 133–143.
- [56] Vlastislav Dohnal, Claudio Gennaro, Pasquale Savino, and Pavel Zezula. 2003. D-index: Distance searching index for metric data sets. *Multimedia Tools and Applications* 21, 1 (2003), 9–33.
- [57] Vlastislav Dohnal, Claudio Gennaro, and Pavel Zezula. 2003. Similarity join in metric spaces using eD-index. In *Proceedings of the International Conference on Database and Expert Systems Applications*. 484–493.
- [58] Karina Figueroa, Edgar Chávez, Gonzalo Navarro, and Rodrigo Paredes. 2006. On the least cost for proximity searching in metric spaces. In *Proceedings of the International Workshop on Experimental and Efficient Algorithms*. 279–290.
- [59] Karina Figueroa, Edgar Chávez, Gonzalo Navarro, and Rodrigo Paredes. 2010. Speeding up spatial approximation search in metric spaces. *Journal of Experimental Algorithmics* 14, 6 (2010), 3–6.
- [60] Karina Figueroa and Nora Reyes. 2019. Permutation’s signatures for proximity searching in metric spaces. In *Proceedings of the International Conference on Similarity Search and Applications*. 151–159.
- [61] Maximilian Franzke, Tobias Emrich, Andreas Züfle, and Matthias Renz. 2016. Indexing multi-metric data. In *Proceedings of the 2016 IEEE 32nd International Conference on Data Engineering*. 1122–1133.
- [62] Kimmo Fredriksson. 2005. Exploiting distance coherence to speed up range queries in metric indexes. *Information Processing Letters* 95, 1 (2005), 287–292.
- [63] Kimmo Fredriksson. 2007. Engineering efficient metric indexes. *Pattern Recognition Letters* 28, 1 (2007), 75–84.
- [64] Ada Wai-chee Fu, Polly Mei-shuen Chan, Yin-Ling Cheung, and Yiu Sang Moon. 2000. Dynamic vp-tree indexing for n-nearest neighbor search given pair-wise distances. *The VLDB Journal* 9, 2 (2000), 154–173.
- [65] Jonathan Goldstein and Raghu Ramakrishnan. 2000. Contrast plots and p-sphere trees: Space vs. time in nearest neighbour searches. In *Proceedings of the 26th International Conference on Very Large Data Bases*. 429–440.
- [66] Magnus Lie Hetland. 2009. The basic principles of metric indexing. In *Proceedings of the Swarm Intelligence for Multi-objective Problems in Data Mining*. 199–232.
- [67] Magnus Lie Hetland. 2015. Ptolemaic indexing. *Journal of Computational Geometry* 6, 1 (2015), 165–184.
- [68] Magnus Lie Hetland, Tomáš Skopal, Jakub Lokoč, and Christian BEECKS. 2013. Ptolemaic access methods: Challenging the reign of the metric space model. *Information Systems* 38, 7 (2013), 989–1006.
- [69] Gisli R. Hjaltason and Hanan Samet. 2003. Index-driven similarity search in metric spaces. *ACM Transactions on Database Systems* 28, 4 (2003), 517–580.
- [70] Michael E. Houle. 2013. Dimensionality, discriminability, density and distance distributions. In *Proceedings of the IEEE 13th International Conference on Data Mining Workshops*. 468–473.
- [71] Michael E. Houle. 2017. Local intrinsic dimensionality I: An extreme-value-theoretic foundation for similarity applications. In *Proceedings of the International Conference on Similarity Search and Applications*. 64–79.
- [72] Michael E. Houle. 2017. Local intrinsic dimensionality II: Multivariate analysis and distributional support. In *Proceedings of the International Conference on Similarity Search and Applications*. 80–95.
- [73] Michael E. Houle. 2020. Local intrinsic dimensionality III: Density and Similarity. In *Proceedings of the International Conference on Similarity Search and Applications*. 248–260.
- [74] Michael E. Houle and Michael Nett. 2013. Rank cover trees for nearest neighbor search. In *Proceedings of the International Conference on Similarity Search and Applications*. 16–29.

- [75] Masahiro Ishikawa, Hanxiong Chen, Kazutaka Furuse, Jeffrey Xu Yu, and Nobuo Ohbo. 2000. MB+ tree: A dynamically updatable metric index for similarity search. In *Proceedings of the International Conference on Web-Age Information Management*. 356–374.
- [76] Hosagrahar V. Jagadish, Beng Chin Ooi, Kian-Lee Tan, Cui Yu, and Rui Zhang. 2005. iDistance: An adaptive B+-tree based indexing method for nearest neighbor search. *ACM Transactions on Database Systems* 30, 2 (2005), 364–397.
- [77] Shichao Jin, Okhee Kim, and Wenya Feng. 2013. M^X -tree: A double hierarchical metric index with overlap reduction. In *Proceedings of the International Conference on Computational Science and Its Applications*. 574–589.
- [78] Iraj Kalantari and Gerard McDonald. 1983. A data structure and an algorithm for the nearest point problem. *IEEE Transactions on Software Engineering* 9, 5 (1983), 631–634.
- [79] Jongik Kim and Hongrae Lee. 2012. Efficient exact similarity searches using multiple token orderings. In *Proceedings of the 2012 IEEE 28th International Conference on Data Engineering*. 822–833.
- [80] Zineddine Kouahla. 2011. Exploring intersection trees for indexing metric spaces. In *Proceedings of the CIAA*.
- [81] Elizaveta Levina and Peter Bickel. 2001. The earth mover's distance is the mallows distance: Some insights from statistics. In *Proceedings of the 8th IEEE International Conference on Computer Vision*. 251–256.
- [82] King-Ip Lin, Hosagrahar V. Jagadish, and Christos Faloutsos. 1994. The TV-tree: An index structure for high-dimensional data. *The VLDB Journal* 3, 4 (1994), 517–542.
- [83] Bing Liu, Wei Wang, Heping Yan, Baile Shi, et al. 2006. A bottom-up distance-based index tree for metric space. In *Proceedings of the 2006 2nd International Conference on Information & Communication Technologies*, Vol. 2. 2929–2934.
- [84] Jakub Lokoč, Juraj Moško, Přemysl Čech, and Tomáš Skopal. 2014. On indexing metric spaces using cut-regions. *Information Systems* 43, July (2014), 1–19.
- [85] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45, September (2014), 61–68.
- [86] Rui Mao, Willard L. Miranker, and Daniel P. Miranker. 2012. Pivot selection: Dimension reduction for distance-based indexing. *Journal of Discrete Algorithms* 13, May (2012), 32–46.
- [87] Mauricio Marin, Roberto Uribe, and Ricardo Barrientos. 2007. Searching and updating metric space databases using the parallel EGNAT. In *Proceedings of the International Conference on Computational Science*. 229–236.
- [88] José Martínez and Zineddine Kouahla. 2012. Indexing metric spaces with nested forests. In *Proceedings of the International Conference on Database and Expert Systems Applications*. 458–465.
- [89] Vladimir Mic, David Novak, and Pavel Zezula. 2017. Sketches with unbalanced bits for similarity search. In *Proceedings of the International Conference on Similarity Search and Applications*. 53–63.
- [90] Luisa Micó, José Oncina, and Rafael C. Carrasco. 1996. A fast branch & bound nearest neighbour classifier in metric spaces. *Pattern Recognition Letters* 17, 7 (1996), 731–739.
- [91] María Luisa Micó, José Oncina, and Enrique Vidal. 1994. A new version of the nearest-neighbour approximating and eliminating search algorithm (AESAs) with linear preprocessing time and memory requirements. *Pattern Recognition Letters* 15, 1 (1994), 9–17.
- [92] Hisham Mohamed and Stéphane Marchand-Maillet. 2015. Quantized ranking for permutation-based indexing. *Information Systems* 52, August - September (2015), 163–175.
- [93] Anirban Mondal, Ayaan Kakkar, Nilesch Padharia, and Mukesh Mohania. 2021. Efficient indexing of top- k entities in systems of engagement with extensions for geo-tagged entities. *Data Science and Engineering* 6, 4 (2021), 411–433.
- [94] Juraj Moško, Jakub Lokoč, and Tomáš Skopal. 2011. Clustered pivot tables for I/O-optimized similarity search. In *Proceedings of the 4th International Conference on Similarity Search and Applications*. 17–24.
- [95] Bilegsaikhan Naidan, Leonid Boytsov, and Eric Nyberg. 2015. Permutation search methods are efficient, yet faster search is possible. *PVLDB* 8, 12 (2015), 1618–1629.
- [96] Alexandros Nanopoulos, Yannis Theodoridis, and Yannis Manolopoulos. 2001. C2P: Clustering based on closest pairs. In *Proceedings of the VLDB*. 331–340.
- [97] Gonzalo Navarro. 1999. Searching in metric spaces by spatial approximation. In *Proceedings of the SPIRE*. 141–148.
- [98] Gonzalo Navarro. 2002. Searching in metric spaces by spatial approximation. *VLDB Journal* 11, 1 (2002), 28–46.
- [99] Gonzalo Navarro, Rodrigo Paredes, Nora Reyes, and Cristian Bustos. 2017. An empirical evaluation of intrinsic dimension estimators. *Information Systems* 64, March (2017), 206–218.
- [100] Gonzalo Navarro and Nora Reyes. 2001. Dynamic spatial approximation trees. In *Proceedings of the 21st International Conference of the Chilean Computer Science Society*. 213–222.
- [101] Gonzalo Navarro and Nora Reyes. 2002. Fully dynamic spatial approximation trees. In *Proceedings of the International Symposium on String Processing and Information Retrieval*. 254–270.
- [102] Gonzalo Navarro and Nora Reyes. 2003. Improved deletions in dynamic spatial approximation trees. In *Proceedings of the 23rd International Conference of the Chilean Computer Science Society*. 13–22.
- [103] Gonzalo Navarro and Nora Reyes. 2009. Dynamic spatial approximation trees for massive data. In *Proceedings of the 2009 Second International Workshop on Similarity Search and Applications*. 81–88.

- [104] Gonzalo Navarro and Nora Reyes. 2016. New dynamic metric indices for secondary memory. *Information Systems* 59, July (2016), 48–78.
- [105] Gonzalo Navarro and Roberto Uribe-Paredes. 2011. Fully dynamic metric access methods based on hyperplane partitioning. *Information Systems* 36, 4 (2011), 734–747.
- [106] Hartmut Noltemeier, Knut Verbarg, and Christian Zirkelbach. 1992. Monotonous bisector* trees—a tool for efficient partitioning of complex scenes of geometric objects. In *Proceedings of Data Structures and Efficient Algorithms, Final Report on DFG Special Joint Initiative*. 186–203.
- [107] David Novak, Michal Batko, and Pavel Zezula. 2011. Metric index: An efficient and scalable solution for precise and approximate similarity search. *Information Systems* 36, 4 (2011), 721–733.
- [108] David Novak and Pavel Zezula. 2016. PPP-codes for large-scale similarity searching. In *Proceedings of the Transactions on Large-Scale Data and Knowledge-Centered Systems XXIV*. 61–87.
- [109] Alexander Ocsa, Carlos Bedregal, and Ernesto Cuadros-Vargas. 2007. A new approach for similarity queries using neighborhood graphs. In *Proceedings of the Brazilian Symposium on Databases*. 131–142.
- [110] Rodrigo Paredes and Edgar Chávez. 2005. Using the k -nearest neighbor graph for proximity searching in metric spaces. In *Proceedings of the International Symposium on String Processing and Information Retrieval*. 127–138.
- [111] Oscar Pedreira and Nieves R. Brisaboa. 2007. Spatial selection of sparse pivots for similarity search in metric spaces. In *Proceedings of the International Conference on Current Trends in Theory and Practice of Computer Science*. 434–445.
- [112] Vladimir Pestov. 2012. Indexability, concentration, and VC theory. *Journal of Discrete Algorithms* 13 (2012), 2–18.
- [113] Ives Rene Venturini Pola, Caetano Traina, and Agma Juci Machado Traina. 2007. The MM-tree: A memory-based metric tree without overlap between nodes. In *Proceedings of the East European Conference on Advances in Databases and Information Systems*. 157–171.
- [114] Jianbin Qin, Wei Wang, Yifei Lu, Chuan Xiao, and Xuemin Lin. 2011. Efficient exact edit similarity query processing with the asymmetric signature scheme. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*. 1033–1044.
- [115] D. A. Rachkovskij. 2017. Distance-based index structures for fast similarity search. *Cybernetics and Systems Analysis* 53, 4 (2017), 636–658.
- [116] Humberto Razente and Maria Camila Nardini Barioni. 2019. Storing data once in M-tree and PM-tree. In *Proceedings of the International Conference on Similarity Search and Applications*. 18–31.
- [117] Humberto Razente, Régis Michel Santos Sousa, and Maria Camila Nardini Barioni. 2018. Metric indexing assisted by short-term memories. In *Proceedings of the International Conference on Similarity Search and Applications*. 107–121.
- [118] Yossi Rubner, Carlo Tomasi, and Leonidas J. Guibas. 2000. The earth mover’s distance as a metric for image retrieval. *International Journal of Computer Vision* 40, 2 (2000), 99–121.
- [119] Enrique Vidal Ruiz. 1986. An algorithm for finding nearest neighbours in (approximately) constant average time. *Pattern Recognition Letters* 4, 3 (1986), 145–157.
- [120] Guillermo Ruiz, Francisco Santoyo, Edgar Chávez, Karina Figueroa, and Eric Sadit Tellez. 2013. Extreme pivots for faster metric indexes. In *Proceedings of the International Conference on Similarity Search and Applications*. 115–126.
- [121] Khalil Al Ruqeishi and Michal Konečný. 2015. Regrouping metric-space search index for search engine size adaptation. In *Proceedings of the International Conference on Similarity Search and Applications*. 271–282.
- [122] Uri Shaft and Raghu Ramakrishnan. 2006. Theory of nearest neighbors indexability. *ACM Transactions on Database Systems* 31, 3 (2006), 814–838.
- [123] Larissa Capobianco Shimomura and Daniel S. Kaster. 2019. HGraph: A connected-partition approach to proximity graphs for similarity search. In *Proceedings of the International Conference on Database and Expert Systems Applications*. 106–121.
- [124] Larissa Capobianco Shimomura, Marcos R. Vieira, and Daniel S. Kaster. 2018. Performance analysis of graph-based methods for exact and approximate similarity search in metric spaces. In *Proceedings of the International Conference on Similarity Search and Applications*. 18–32.
- [125] Eliezer Silva, Thiago Teixeira, George Teodoro, and Eduardo Valle. 2014. Large-scale distributed locality-sensitive hashing for general metric data. In *Proceedings of the International Conference on Similarity Search and Applications*. 82–93.
- [126] Tomáš Skopal and David Hoksza. 2007. Improving the performance of M-tree family by nearest neighbor graphs. In *Proceedings of the East European Conference on Advances in Databases and Information Systems*. 172–188.
- [127] Tomáš Skopal and Jakub Lokoč. 2008. NM-tree: Flexible approximate similarity search in metric and non-metric spaces. In *Proceedings of the International Conference on Database and Expert Systems Applications*. 312–325.
- [128] Tomáš Skopal and Jakub Lokoč. 2009. New dynamic construction techniques for M-tree. *Journal of Discrete Algorithms* 7, 1 (2009), 62–77.
- [129] Tomáš Skopal, Jaroslav Pokorný, Michal Krátký, and Václav Snášel. 2003. Revisiting M-tree building principles. In *Proceedings of the East European Conference on Advances in Databases and Information Systems*. 148–162.

- [130] Tomáš Skopal, Jaroslav Pokorný, and Vaclav Snasel. 2004. PM-tree: Pivoting metric tree for similarity search in multimedia databases. In *Proceedings of the East European Conference on Advances in Databases and Information Systems*. 803–815.
- [131] Michael Stonebraker and Uğur Çetintemel. 2018. “One size fits all”: An idea whose time has come and gone. In *Proceedings of the Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*. 441–462.
- [132] Michael Stonebraker, Samuel Madden, Daniel J. Abadi, Stavros Harizopoulos, Nabil Hachem, and Pat Helland. 2018. The end of an architectural era: It’s time for a complete rewrite. In *Proceedings of the Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*. 463–489.
- [133] Richard C. Tillquist and Manuel E. Lladser. 2019. Low-dimensional representation of genomic sequences. *Journal of Mathematical Biology* 79, 1 (2019), 1–29.
- [134] Ken Tokoro, Kazuaki Yamaguchi, and Sumio Masuda. 2006. Improvements of TLAESA nearest neighbour search algorithm and extension to approximation search. In *Proceedings of the 29th Australasian Computer Science Conference*. 77–83.
- [135] Caetano Traina, Agma Traina, Bernhard Seeger, and Christos Faloutsos. 2000. Slim-trees: High performance metric trees minimizing overlap between nodes. In *Proceedings of the International Conference on Extending Database Technology*. 51–65.
- [136] Caetano Traina, Agma J. M. Traina, Marcos R. Vieira, and Christos Faloutsos. 2007. The omni-family of all-purpose access methods: A simple and effective way to make similarity search more efficient. *The VLDB Journal* 16, 4 (2007), 483–505.
- [137] Caetano Traina Jr, Agma Traina, Roberto Santos Filho, and Christos Faloutsos. 2002. How to improve the pruning ability of dynamic metric access methods. In *Proceedings of the 11th International Conference on Information and Knowledge Management*. 219–226.
- [138] Jeffrey K. Uhlmann. 1991. Metric trees. *Applied Mathematics Letters* 4, 5 (1991), 61–62.
- [139] Jeffrey K. Uhlmann. 1991. Satisfying general proximity/similarity queries with metric trees. *Information Processing Letters* 40, 4 (1991), 175–179.
- [140] Lucia Vadicamo, Richard Connor, Fabrizio Falchi, Claudio Gennaro, and Fausto Rabitti. 2019. Splx-perm: A novel permutation-based representation for approximate metric search. In *Proceedings of the International Conference on Similarity Search and Applications*. 40–48.
- [141] Reinier H. Van Leuken and Remco C. Veltkamp. 2011. Selecting vantage objects for similarity indexing. *ACM Transactions on Multimedia Computing, Communications, and Applications* 7, 3 (2011), 1–18.
- [142] Marcos R. Vieira, Caetano Traina Jr, Fabio J. T. Chino, and Agma J. M. Traina. 2004. DBM-tree: A dynamic metric access method sensitive to local density data. In *Proceedings of the SBBD*. 163–177.
- [143] Marcos R. Vieira, Caetano Traina Jr, Fabio J. T. Chino, and Agma J. M. Traina. 2010. DBM-tree: A dynamic metric access method sensitive to local density data. *Journal of Information and Data Management* 1, 1 (2010), 111–111.
- [144] Juan Miguel Vilar. 1995. Reducing the overhead of the AESA metric-space nearest neighbour searching algorithm. *Information Processing Letters* 56, 5 (1995), 265–271.
- [145] Ilya Volnyansky and Vladimir Pestov. 2009. Curse of dimensionality in pivot based indexes. In *Proceedings of the International Workshop on Similarity Search and Applications*. 39–46.
- [146] Roger Weber, Hans-Jörg Schek, and Stephen Blott. 1998. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *Proceedings of the VLDB*. 194–205.
- [147] Xiaojing Xie, Jihong Guan, and Shuigeng Zhou. 2015. Similarity evaluation of DNA sequences based on frequent patterns and entropy. In *Proceedings of the BMC Genomics*, Vol. 16. 1–10.
- [148] Yuki Yamagishi, Kazuo Aoyama, Kazumi Saito, and Tetsuo Ikeda. 2018. Pivot generation algorithm with a complete binary tree for efficient exact similarity search. *IEICE Transactions on Information and Systems* 101, 1 (2018), 142–151.
- [149] Peter N Yianilos. 1993. Data structures and algorithms for nearest neighbor. In *Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms*, Vol. 66. 311.
- [150] Peter N. Yianilos. 1999. Excluded middle vantage point forests for nearest neighbor search. In *Proceedings of the ALENEX*.
- [151] Pavel Zezula, Giuseppe Amato, Vlastislav Dohnal, and Michal Batko. 2006. *Similarity Search: The Metric Space Approach*. Springer Science & Business Media.
- [152] Pavel Zezula, Pasquale Savino, Giuseppe Amato, and Fausto Rabitti. 1998. Approximate similarity retrieval with M-trees. *The VLDB Journal* 7, 4 (1998), 275–293.
- [153] Ming Zhang and Reda Alhajj. 2010. Effectiveness of NAQ-tree as index structure for similarity search in high-dimensional metric space. *Knowledge and Information Systems* 22, 1 (2010), 1–26.
- [154] Zhenjie Zhang, Marios Hadjieleftheriou, Beng Chin Ooi, and Divesh Srivastava. 2010. Bed-tree: An all-purpose index structure for string similarity search based on edit distance. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*. 915–926.

- [155] Xiangmin Zhou, Guoren Wang, Jeffrey Xu Yu, and Ge Yu. 2003. M^+ -tree: A new dynamical multidimensional index for metric spaces. In *Proceedings of the 14th Australasian Database Conference*. 161–168.
- [156] Xiangmin Zhou, Guoren Wang, Xiaofang Zhou, and Ge Yu. 2005. BM^+ -tree: A hyperplane-based index method for high-dimensional metric spaces. In *Proceedings of the International Conference on Database Systems for Advanced Applications*. 398–409.

Received 21 November 2020; revised 11 January 2022; accepted 2 April 2022